

Open-World Task and Motion Planning via Vision-Language Model Generated Constraints

Nishanth Kumar^{*1,2}, William Shen^{1,2}, Fabio Ramos¹, Dieter Fox^{1,2},
Tomás Lozano-Pérez², Leslie Pack Kaelbling² and Caelan Reed Garrett¹

¹NVIDIA Research, ²MIT CSAIL

Abstract: Foundation models like Vision-Language Models (VLMs) excel at common sense vision and language tasks such as visual question answering. However, they cannot yet directly solve complex, long-horizon robot manipulation problems requiring precise continuous reasoning. Task and Motion Planning (TAMP) systems can handle long-horizon reasoning through discrete-continuous hybrid search over parameterized skills, but rely on detailed environment models and cannot interpret novel human objectives, such as arbitrary natural language goals. We propose integrating VLMs into TAMP systems by having them generate discrete and continuous language-parameterized *constraints* that enable open-world reasoning. Specifically, we use VLMs to generate discrete action ordering constraints that constrain TAMP’s search over action sequences, and continuous constraints in the form of code that augments traditional TAMP manipulation constraints. Experiments show that our approach, OWL-TAMP, outperforms baselines relying solely on TAMP or VLMs across several long-horizon manipulation tasks specified directly in natural language. We additionally demonstrate that OWL-TAMP can be deployed with an off-the-shelf TAMP system to solve challenging manipulation tasks on real-world hardware.

1 Introduction

Solving complex, long-horizon manipulation tasks is a fundamental challenge in robotics. Recently, there has been significant interest in solving such tasks directly from language and image input [1, 2, 3]. In this setting, a user provides a command like “put away all the cutlery in the utensil holder” and the robot must execute a sequence of actions to achieve the intended effect. Even with pre-engineered manipulation controller primitives, such as those for moving, picking objects, and placing objects, these problems remain difficult: the robot must correctly interpret the command and sequence actions with the correct discrete arguments, such as which objects to grasp, and continuous arguments, such as grasps, stable placement poses and collision-free motions.

Consider the task in Figure 1, where the robot must “put the banana near where the other fruits are initially.” One approach is to prompt a Vision-Language Model (VLM) [4] with the initial image, task, and available controllers [5]. VLMs excel at interpreting commands and leveraging common sense, and would likely suggest picking the banana before placing it. However, they struggle to reason about continuous parameter values and geometry (e.g., synthesizing collision-free grasps and placements) without task-specific fine-tuning [6, 2]. Here, the robot’s gripper cannot safely grasp the banana without first moving obstructing objects. By contrast, classical Task and Motion Planning (TAMP) systems explicitly reason over discrete and continuous values using symbolic predicates, operators, and geometric models [7, 8]. A TAMP system would detect the blocked grasp and plan to move the milk carton before picking the banana. However, TAMP systems require extensive manual modeling and cannot handle novel concepts without additional engineering. In this case, the TAMP

^{*}Work conducted partially during an internship at NVIDIA Research. Correspondence to njk@csail.mit.edu, cgarrett@nvidia.com.

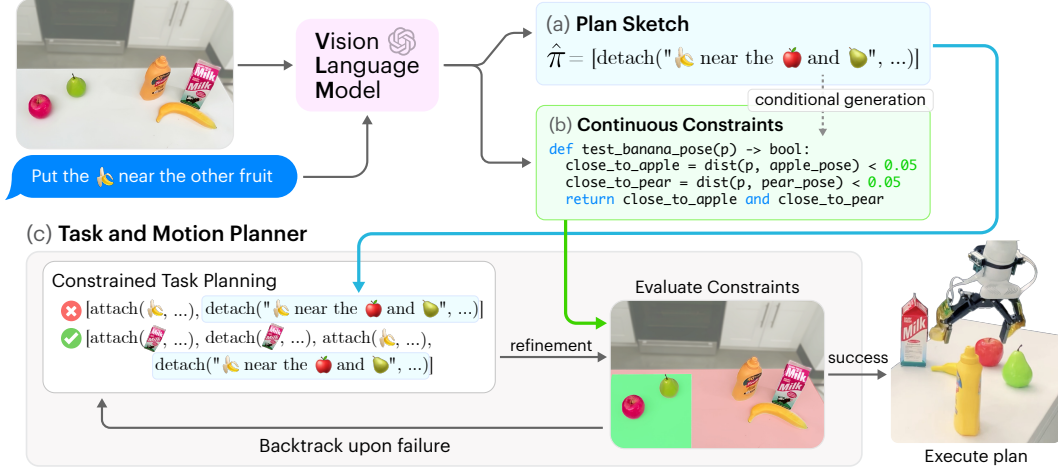


Figure 1: **OWL-TAMP Overview.** OWL-TAMP uses VLMs to generate task-specific open-world constraints that augment traditional robot constraints like kinematics and grasp stability. The VLM first generates (a) plan sketches, open-world actions with natural language descriptions, and uses these to generate (b) continuous constraints in the form of code that check whether actions achieve their intended open-world predicate effects. (c) A TAMP system consumes these constraints to solve for a plan that satisfies both traditional and VLM-generated constraints. Here, OWL-TAMP moves the milk carton, enabling the banana to be grasped successfully.

system lacks the concept for “near”, and thus cannot place the banana correctly “near” the other fruits.

In this work, we combine the strengths of foundation models and TAMP to solve manipulation tasks specified in natural language. Our key insight is that we can leverage the commonsense and language-grounding capabilities of VLMs to guide TAMP systems through dynamically generated *constraints*. Specifically, VLMs map language into both code expressing continuous action constraints (e.g., valid poses “near” other fruits) and discrete constraints *plan sketches* (e.g., pick the banana, then place it “near” the other fruits). These constraints are combined with common, pre-existing TAMP constraints like collision avoidance and kinematics, and are used by a TAMP system to solve for a plan that jointly satisfies all constraints. In our example, the TAMP system produces a correct plan that moves the milk carton to enable grasping the banana, then places it near the other fruit as dictated by the VLM-generated continuous pose constraint. By integrating such constraints, TAMP systems become “open-world” – capable of solving tasks beyond their pre-defined symbolic vocabulary (e.g., reasoning about “near” or “oriented straight” without explicit pre-defined predicates). As a result, our approach is able to solve tasks beyond the closed-world scope of prior work that just use VLMs to accelerate discrete planning [9].

We propose OWL-TAMP (Open-World Language-based TAMP), our approach that integrates open-world concepts into TAMP via constraint generation over actions and continuous variables. Our main contribution is to propose a clear contract — in the form of *constraints* — for combining VLMs with generic domain-independent TAMP systems to enable zero-shot generalization to a variety of tasks. More specifically, OWL-TAMP: (1) generates constraints on action sequences from language to specify plan sketches; (2) generates constraints on continuous variables within these plans via code; and (3) integrates both into standard TAMP pipelines. In simulation, OWL-TAMP achieves higher success rates on open-world tasks than several baselines, including pure VLM and pure TAMP approaches. We also demonstrate that it enables a real robot to solve complex, natural-language based long-horizon tasks with initial states given as raw camera images.

2 Related Work

Task and Motion Planning (TAMP). A standard approach for solving long-horizon robot tasks via joint discrete-continuous reasoning is TAMP [10, 11]. TAMP can solve tasks expressed in natural

language by translating language into symbolic goals [12], but assumes (1) exact translation into a pre-defined predicate set and (2) that these goals are achievable with existing symbolic operators. For instance, solving tasks like Figure 1 would require a “near” predicate; if only an “on top” predicate exists, translation would fail. Expanding TAMP’s capabilities thus requires manual or learned predicate invention [13, 14, 15, 16]. In contrast, our approach uses pretrained VLMs without fine-tuning to generate constraints that expand TAMP’s capabilities without task-specific demonstration or exploration data (Section 6).

LLMs, VLMs and VLAs for Robotic Manipulation. Recent work has leveraged VLMs [17, 18, 1, 19] to enable complex short-horizon visuomotor behaviors, such as pouring water into a cup. In contrast, we focus on longer horizon tasks composed of multiple atomic behaviors. Additionally, to the extent these approaches do handle multi-step tasks (e.g., pick-and-place tasks), they often make assumptions or leverage heuristics specific to the particular class of tasks — such as hardcoding the action ordering [1] or learning in simulation — and are thus not zero-shot [19]. More recent works attempt to solve long-horizon tasks by training Vision Language Action models (VLAs) on large amounts of robot demonstration data [6, 20]. However, these models require finetuning on some task and environment-specific data when adapting to novel setups and embodiments. By contrast, our approach combines VLMs with generic, domain-independent TAMP systems to solve tasks with no requirement for additional demonstrations or learning in simulation.

LLMs and VLMs for Robot Task Planning. Another line of work uses foundation models for planning by sequencing discrete skills in settings where continuous parameters are either unnecessary or easily determined by heuristics [21, 22, 23, 24]. Some works address continuous parameters [24, 25, 26, 3, 27, 28, 29], but rely on foundation models to predict them directly or generate code, typically succeeding only when parameters are easily inferred from images or text (e.g., top-down grasping). A few approaches tackle complex tasks requiring precise discrete and continuous parameters [30, 31, 32, 2], but struggle with discrete-level *backtracking* over action sequences or long-horizon tasks where explicit search is preferable [33]. Our method instead combines VLMs with off-the-shelf TAMP systems to handle long-horizons and more complex constraints. Recent work [9] leverages VLMs to generate subgoals for a TAMP system to solve sequentially, but does not generate any continuous constraints. Concurrent work [34] also defines a constraint-based contract between LLMs and TAMP, but only handles discrete constraints and requires a custom TAMP solver.

Overall, while there are prior works that explore the general idea of combining foundation models with search-based planning, our approach is the *first* to directly integrate with full off-the-shelf TAMP systems. In OWL-TAMP, the foundation model is not simply accelerating search but rather writing open-world continuous and discrete constraints to augment the model that search is being performed on.

3 Problem Formulation

We aim to solve tasks specified in natural language, given an initial scene observation, a set of objects, a library of manipulation controllers, and a pre-existing Task and Motion Planning (TAMP) system with symbolic components. We define a task as a tuple $\langle s_0, \mathcal{A}, \mathcal{I}, L \rangle$, where s_0 is the initial state of the robot and a set of named objects (identified via object detection and segmentation) in the environment (e.g., joint positions, object poses obtained from vision-based pose estimation), $\mathcal{A}$ is a set of *parameterized actions*, $\mathcal{I}$ is a set of initial image observations of the scene, and L is the natural language instruction (e.g., “put the banana near the other fruit” in Figure 1).

Each parameterized action in $\mathcal{A}$ consists of a list of typed parameters, which may be discrete (e.g., object identifiers) or continuous (e.g., object poses or robot configurations), and a low-level controller (e.g., a joint-trajectory controller) that executes the action according to the parameter values. For example, consider the typed parameters of an *attach* action: `attach(o: obj, p: pose, g: grasp, q: conf)`. Here, o is a discrete object identifier, $p \in \text{SE}(3)$ is an approach pose near object o , $g \in \text{SE}(3)$ is a grasp pose where the robot closes its fingers,

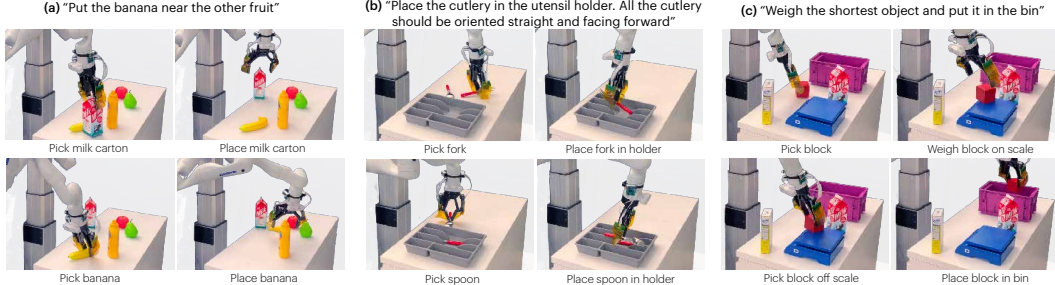


Figure 2: **Real-World Tasks.** OWL-TAMP (a) reasons to move the milk obstructing the banana, then places the banana near the other fruit (Figure 1). (b) plans placements satisfying both the language goal and collision constraints. (c) identifies the block as shortest, and weighs it on the scale before disposing.

and $q \in \mathbb{R}^d$ is the corresponding robot joint configuration used to execute grasp g on o . Specifying concrete values for all parameters *grounds* the action, yielding a fully executable manipulation controller.

The robot’s objective is to generate and execute a *plan*: a sequence of grounded actions which, when applied from the initial state s_0 , results in a final state s_* that satisfies the goal described by L .

3.1 Task and Motion Planning

Solving the tasks described above is challenging because the robot must not only select a valid sequence of actions, but also ground each action with precise parameters that achieve the goal described by L . We are interested in leveraging a TAMP system to help address these challenges; we thus assume access to a model-based TAMP system that is fixed and shared across all tasks.

Within TAMP, we model states and actions using a Planning Domain Definition Language (PDDL)-style [35] factored action language, where states are sets of true-valued literals, i.e., *predicates* applied to specific parameters. For example, the initial state s_0 in Figure 1 includes $[\text{AtConf}(q_0), \text{HandEmpty}(), \text{AtPose}(\text{banana}, p_0), \text{On}(\text{banana}, \text{table}), \dots]$, indicating that the robot is at configuration q_0 , its gripper is empty, the banana is at placement pose p_0 and on the table.

Each parameterized action a in $\mathcal{A}$, defined as part of the task, is extended in the TAMP system with a set of constraints (**con**) that the parameters must satisfy, preconditions (**pre**) that must hold before applying the action, and effects (**eff**) that describe how the state changes after executing action a . The actions *move* and *attach* model the robot moving between two configurations and attaching an object to itself via grasping, respectively.

```

move( $q_1 : \text{conf}, q_2 : \text{conf}, \tau : \text{traj}$ )
  con: [ $\text{Motion}(q_1, \tau, q_2)$ ]
  pre: [ $\text{AtConf}(q_1)$ ]
  eff: [ $\text{AtConf}(q_2), \neg \text{AtConf}(q_1)$ ]
attach( $o : \text{obj}, p : \text{pose}, g :$ 
grasp,  $q : \text{conf}$ )
  con: [ $\text{Kin}(q, o, g, p)$ ]
  pre: [ $\text{AtPose}(o, p), \text{HandEmpty}(), \text{AtConf}(q)$ ]
  eff: [ $\text{AtGrasp}(o, g), \neg \text{AtPose}(o, p), \neg \text{HandEmpty}()$ ]

```

The $\text{Motion}(q_1, \tau, q_2)$ constraint checks that τ is a valid trajectory connecting configurations q_1 and q_2 , while $\text{Kin}(q, o, g, p)$ checks whether configuration q satisfies a kinematic constraint for grasping object o at placement pose p with grasp g . The $\text{AtGrasp}(o, g)$ predicate indicates that object o is currently grasped with grasp pose g . Additionally, for some of these constraints, we have standard robotics samplers that generate continuous values that satisfy these constraints, namely an inverse kinematics solver for Kin and motion planner for Motion . The challenge in planning is *refinement*: generating input parameter values for these actions (e.g. the pose p and grasp g for *attach*).

The goal in TAMP is typically specified by a symbolic goal G , a set of literals that must hold in the final state. The planner searches for a sequence of grounded actions that results in a state s_* where all literals in G evaluate to true. This search is conducted within a 3D model of the environment and objects: in real-world environments, this is constructed from the robot’s initial sensory observations such as point clouds. A key feature of TAMP is that the planner is capable of not just considering a single action sequence to achieve the goal (e.g., directly `attach`-ing then `detach`-ing the banana in Figure 1c), but rather exploring multiple sequences (via *backtracking* over an abstract plan) across multiple possible action sequences (e.g., first moving the milk carton out of the way) until it finds one that accomplishes the task.

4 Open-World Actions and Predicates

Traditional TAMP systems rely on a fixed set of symbolic predicates and operators, which limits their applicability to tasks that can be fully specified within this vocabulary. However, many open-ended tasks in unstructured environments involve object semantics, spatial relationships, or context-dependent goals that are not explicitly encoded in the domain definition. To address this, we propose a new abstraction that extends TAMP representations through *open-world actions*, which augment standard TAMP actions with natural language parameters, and *open-world predicates*, whose behavior is dynamically grounded via natural language.

Open-World Actions. An open-world action extends a parameterized TAMP action (Section 3.1) with an additional discrete argument: a natural language description d as a string. This description specifies task-specific intent that constrains the set of valid continuous parameter values, such as object poses or robot configurations. Open-world actions enable novel behaviors that go beyond what can be expressed using the fixed symbolic vocabulary of a traditional TAMP system. For example, in Figure 1, the `detach` action is invoked with the description `detach(“place the banana near the apple and pear”, ...)`, which narrows the feasible placement region from the entire table to a localized region, as visualized by the colored regions. A different description, such as “place the banana atop the orange” would restrict the final pose of the banana to be stably resting on the orange.

Open-World Predicates. To enforce that the intent expressed by the language description d is achieved, we introduce open-world predicates, which are appended to the constraint list (**con**) of an open-world action. Traditional predicates rely on hand-engineered, fixed classifiers to evaluate whether a *literal* — a predicate with fully specified parameter values — is considered true or false. In contrast, an open-world predicate uses a classifier that is generated dynamically at planning time, conditioned on the description d and other parameters (Figure 1b). We assume that these predicates take in d as a parameter, along with any other objects and continuous parameters, and output a boolean value indicating that the parameters are set such that d is achieved. We specifically use a VLM to generate this classifier function in the form of Python code (Section 5.2). The resulting classifiers act as constraints during planning, restricting the set of valid continuous parameters (e.g., object poses) according to whether they satisfy the task-specific intent described by d .

Example. We make the `detach` action open-world by modeling the description d as a parameter and including the open-world predicate `VLMConstraint` in its constraints:

```
detach( $d$ : description,  $o$ : obj,  $g$ : grasp,  $p$ : pose,  $q$ : conf)
  con: [Kin( $q, o, g, p$ ), VLMConstraint( $d, o, p$ )]
  pre: [AtGrasp( $o, g$ ), AtConf( $q$ ),
         $\neg \exists o', p'. \text{AtPose}(o', p') \wedge \text{Collision}(o, p, o', p')$ ]
  eff: [AtPose( $o, p$ ),  $\neg \text{AtGrasp}(o, g)$ , HandEmpty()]
```

Let the open-world predicate `VLMConstraint( $d, o, p$ )` be true if placing object o at placement pose p satisfies description d . This allows the `detach` action to jointly respect both task-specific constraints specified by d and engineered constraints such as kinematic feasibility (`Kin`). In Figure 1c, the TAMP planner rejection samples the set of reachable poses (generated by samplers for `Kin`) against

VLMConstraint, implemented by the VLM-generated classifier function `test_banana_pose`, to yield a collision-free banana placement that is near the other fruit.

The `detach` action serves as a simple illustrative example that operates over a single object. Open-world actions can more generally use any number of objects and parameters, and model a variety of other interaction types. The following `transport` action models moving an object o through pose waypoints p_1 and p_2 to then interact with object o' . The waypoints p_1 and p_2 are determined by a VLM constraint `VLMConstraint2` and can, for example, represent both the start and end of a pouring motion involving a cup (e.g. in the *SoupPour* task in Section 6) into a bowl o' as well as filling a cup o from a faucet o' . The key takeaway is that open-world actions enable planning with both manually engineered TAMP constraints and dynamically generated, task-specific constraints grounded in natural language, expressed as open-world predicates.

```
transport(d: description, o: obj, g: grasp,
         p1: pose, p2: pose, o': obj, q1: conf, q2: conf)
con: [Kin(q1, o, g, p1), Kin(q2, o, g, p2),
      VLMConstraint2(d, o, p1, p2, o')]
pre: [AtGrasp(o, g), AtConf(q1)]
eff: [AtConf(q2), ¬AtConf(q1)]
```

5 Open-World Language-Based TAMP

Given a natural language instruction, images of the initial scene, a set of objects segmented within a scene, a TAMP system, and a collection of open-world actions and predicates, OWL-TAMP produces a valid plan that satisfies both manually engineered symbolic constraints and dynamically generated constraints from a VLM. Our approach operates in three stages: (1) it prompts a VLM to generate a discrete *plan sketch* of open-world actions relevant to the task (Section 5.1); (2) it grounds open-world predicates in the sketch into executable *continuous constraints* via VLM-generated classifier functions (Section 5.2); and (3) it solves the resulting TAMP problem using an off-the-shelf planner (Section 5.3).

5.1 Generating Open-World Action Ordering Constraints

Given the language instruction L and initial images $\mathcal{I}$ (Section 3), we first prompt a VLM to produce a *plan sketch* $\hat{\pi}$: a partial sequence of open-world actions with discrete object arguments and natural language descriptions that reflect how to achieve the task specified by L . This sketch encodes a high-level plan structure, specifying a minimal set of actions and their order for a plan to be valid. In the banana task (Figure 1), the sketch could be:

$\hat{\pi} = [\text{detach}(\text{"place the banana near the apple and pear"}, \text{banana}, \dots)]$.

In the weigh and dispose shortest object task (Figure 2c), the sketch might be:

$\hat{\pi} = [\text{detach}(\text{"weigh block on scale"}, \text{block}, \dots), \text{detach}(\text{"place block in bin"}, \text{block}, \dots)]$,

allowing us to `detach` the same object on different support surfaces. However, VLM-generated sketches are not guaranteed to be complete – they may omit necessary actions (e.g., forgetting to `attach` before `detach`) or propose actions that are infeasible due to low-level constraints such as collisions or kinematic feasibility. In the banana task (Figure 1), the VLM may omit a required `attach(banana, ...)` before `detach`, or fail to consider that the milk carton obstructs all grasps on the banana.

To address these issues, we treat the plan sketch as a set of discrete action-ordering constraints. Specifically, we constrain the TAMP planner to generate plans that include $\hat{\pi}$ as a subsequence – that is, the specified actions must appear in the given order, though additional actions may be inserted before, between, or after them to ensure the plan is feasible.

We generate plan sketches using a few-shot prompting approach with a VLM with no fine-tuning, and describe how they are symbolically encoded into a TAMP system in Appendix A.1.2.

Method	Tasks										Overall
	Berry1	Citrus	Berry2	BerryCook	FruitSort	Coffee	Mug1	Mug2	Mug3	SoupFour	
CaP	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0
CaP-sample	100% $\pm$ 0.0	20% $\pm$ 12.6	20% $\pm$ 12.6	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	14% $\pm$ 3.4
Direct trans.	100% $\pm$ 0.0	100% $\pm$ 0.0	100% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	20% $\pm$ 12.6	0% $\pm$ 0.0	32% $\pm$ 4.7
No sample	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0
No disc.	100% $\pm$ 0.0	100% $\pm$ 0.0	100% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	0% $\pm$ 0.0	20% $\pm$ 12.6	0% $\pm$ 0.0	32% $\pm$ 4.7
No cont.	100% $\pm$ 0.0	100% $\pm$ 0.0	100% $\pm$ 0.0	100% $\pm$ 0.0	10% $\pm$ 9.5	60% $\pm$ 15.5	70% $\pm$ 14.5	0% $\pm$ 0.0	0% $\pm$ 0.0	20% $\pm$ 12.6	56% $\pm$ 5.0
No back.	100% $\pm$ 0.0	90% $\pm$ 9.5	100% $\pm$ 0.0	60% $\pm$ 15.5	80% $\pm$ 12.6	100% $\pm$ 0.0	40% $\pm$ 15.5	30% $\pm$ 14.5	0% $\pm$ 0.0	0% $\pm$ 0.0	60% $\pm$ 4.9
OWL-TAMP	100% $\pm$ 0.0	100% $\pm$ 0.0	100% $\pm$ 0.0	60% $\pm$ 15.5	100% $\pm$ 0.0	100% $\pm$ 0.0	100% $\pm$ 0.0	100% $\pm$ 0.0	70% $\pm$ 14.5	90% $\pm$ 9.5	92% $\pm$ 2.7

Table 1: **Success rates on all tasks.** Entries report the mean success rate over 10 random seeds and the binomial standard error in percentage points. We bold methods that are not significantly worse than the best method in that column under a one-tailed two-proportion z -test ($\alpha=0.05$). The final column pools successes across all 10 tasks ($n=100$).

To enable a VLM to generate a plan sketch for a given task, we associate a natural language description of each available action with that particular action. Although we could directly prompt a VLM for relevant actions and goals, without a list of candidates, the VLM is likely to be syntactically and semantically inaccurate. Instead, we first ground the set of *reachable* actions A and literals Λ (i.e., grounded predicates) available to the TAMP system before prompting the VLM to return values in these sets. We use *relaxed planning* [36] from the initial state s_0 to simultaneously ground [37, 38] and explore the sets of reachable actions A and literals Λ . We use placeholders to instantiate continuous parameters, such as *optimistic* values [39], to ensure a finite set of actions are instantiated. Similarly, we use placeholders for description parameters.

Algorithm 1 VLM Task Reasoning

```

1: procedure VLM-TASK-REASONING( $s_0, \mathcal{A}, \mathcal{I}, L$ )
2:    $A \leftarrow \text{GROUND-ACTIONS}(s_0, \mathcal{A})$ 
3:    $\Lambda \leftarrow s_0 \cup \{l \mid a \in A, l \in a.\text{eff}\}$ 
4:    $[a_1, \dots, a_n, l_1, \dots, l_k] \leftarrow \text{QUERY-VLM}(\text{"What partial plan using actions } \{A\} \text{ for goal literals } \{\Lambda\} \text{ achieves goal } \{L\}?" )$ 
5:   for  $i \in [1, n-1]$  do
6:      $a_i.\text{eff} \leftarrow a_i.\text{eff} \cup \{\text{Executed}(i)\}$ 
7:      $a_{i+1}.\text{pre} \leftarrow a_{i+1}.\text{pre} \cup \{\text{Executed}(i)\}$ 
8:    $a_n.\text{eff} \leftarrow a_n.\text{eff} \cup \{\text{Executed}(n)\}$ 
9:    $G \leftarrow \{l_1, \dots, l_k\}$ 
10:  return SOLVE-TAMP( $s_0, A, G \cup \{\text{Executed}(n)\}$ )

```

Algorithm 1 presents the VLM partial planning pseudocode. It takes in a TAMP problem $\langle s_0, \mathcal{A}, \mathcal{I}, L \rangle$, where L is a text goal description and $\mathcal{I}$ is a set of initial image observations of the scene. It first grounds the set of actions A reachable from s_0 using GROUND-ACTIONS. Then, it accumulates the set of reachable literals Λ by taking the effects of all actions A . Then, it prompts QUERY-VLM for a partial plan $[a_1, \dots, a_n, l_1, \dots, l_k]$ using actions $a_i \in A$ and goal literals $l_j \in \Lambda$ that achieve L . Here, the literals $l_j \in \Lambda$ are intended to be a translation of the goal into atoms, while the sequence of actions is a way of achieving that goal. Importantly, we have the VLM fill in the description parameter d for each of these actions. We transform the original problem to so that solutions admit the partial plan as a subsequence. Specifically, we create a predicate EXECUTED that models whether the i th action in the plan was executed and add EXECUTED to the effects of action a_i and the preconditions of action a_{i+1} . Finally, we make the planning goal $G = \{l_i, \dots, l_m\} \subseteq L$ and EXECUTED(n).

5.2 Grounding Open-World Predicates with a VLM

After obtaining a plan sketch from a VLM, we ground each open-world predicate into a concrete constraint that can be evaluated during planning. These predicates, which appear in the open-world actions within the plan sketch $\hat{\pi}$, are grounded by dynamically generating a *classifier function* for each predicate separately once the predicate is instantiated with a specific natural language descrip-

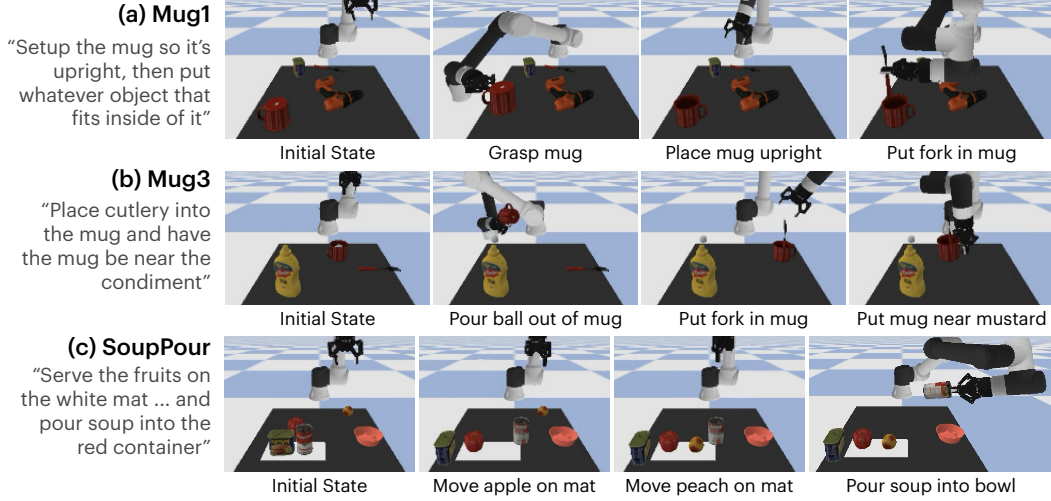


Figure 3: **Simulated RAVENS-YCB Tasks.** OWL-TAMP (a) reorients the mug and infers only the fork fits inside; (b) removes the ball from the mug before putting the fork inside, then moves it closer to the mustard; (c) moves the spam aside to fit fruits on the mat before pouring soup into the bowl.

tion. The resulting classifier operates over continuous-valued arguments and returns a boolean indicating whether a given literal (i.e., a predicate with fully specified arguments), is considered true or false. We refer to these classifier functions as *continuous constraints*.

Recall that the predicate $\text{VLMConstraint}(d, o, p)$ in `detach` is used to constrain the placement of object o based on the language description d . When instantiated with $d =$ “place the banana near the apple and the pear” and $o =$ banana, OWL-TAMP prompts a VLM to generate a Python function `test_banana_pose(p)` that returns true if pose p lies near the apple and pear on the table (Figure 1b). During planning, the TAMP system evaluates this constraint by applying the function to candidate placement poses of the banana. We follow prior work [3, 2] and prompt a VLM separately for each predicate and description using a few-shot template and a code-book of basic helper functions (e.g., for computing bounding boxes or checking collisions between two objects), which the VLM can compose to construct more complex constraints (Appendix A.6).

5.3 Planning with VLM-Generated Constraints

Given the discrete plan sketch and continuous constraints generated by the VLM, we can now solve the TAMP problem via an off-the-shelf planning algorithm, such as SeSaME [11] or PDDL-Stream [8] (see Appendix A.1.1). The planner searches for a valid sequence of discrete actions, constrained by the plan sketch, and solves for continuous parameter assignments satisfying both manually engineered and VLM-generated constraints. In the banana task (Figure 1), the planner first attempts to attach and detach the banana directly, but fails to find collision-free grasps due to the obstructing milk carton. It then backtracks and finds a feasible plan that first moves the milk carton before grasping and placing the banana at a pose that satisfies VLMConstraint and is reachable.

6 Experiments and System Demonstration

Tasks. We evaluate on ten open-world manipulation tasks, spanning a variety of skills including obstacle removal, sorting, reorientation, insertion, and pouring. All tasks implemented in the RAVENS-YCB environment on a UR5 arm, adapted from prior work [2, 40] (see Appendix A.2).

- *Berry1*: “put the strawberry onto the light-grey region at the center of the table”.
- *Citrus*: “pack the citrus fruit onto the plate”.
- *Berry2*: Same as *Berry1*, but requires moving an obstacle out of the way of the light-grey region.

- *BerryCook*: “Cook the strawberry by putting it in the pan, then finally simply place it in the bowl. The strawberry should only be in the bowl at the end!”.
- *FruitSort*: “Put all the fruit to the left of the line bisecting the table”.
- *Coffee*: “I want to pour some coffee into the cup; can you set up the cup on the table so I can do this properly?” (requires reorienting the cup so that it is placed ‘right-side-up’ on the table).
- *Mug1*: “Setup the mug so it’s upright, then put whatever object that fits inside of it”.
- *Mug2*: “Place cutlery inside the mug and then place the mug itself on the table near the condiment” (the mug’s opening is obstructed by a large orange, which must be moved out of the way).
- *Mug3*: Same as *CutleryInMug2*, except that in the initial state, the mug is unobstructed by an orange but instead contains a ball, which must be removed by ‘pouring’ it out.
- *SoupPour*: “Serve the fruits on the white mat (make sure the peach is to the right of the apple” and pour soup into the red container”. The white mat is originally obstructed by the soup can as well as a spam can, which must be moved out of the way to successfully place the fruits.

Approaches. We list the various approaches we compare to OWL-TAMP across various tasks.

- *CaP*: Application of Code as Policies [3], using a VLM to generate plan code directly from initial states. Few-shot examples and helper functions match OWL-TAMP.
- *Direct trans.*: Prompting an LLM to translate natural language goals to symbolic goals (see Section 3.1), then solving with TAMP. This baseline is inspired by [41, 12]
- *CaP-sample*: CaP variant inspired by [2], where the VLM samples multiple continuous values for skills and selects the first set that satisfies robot constraints.
- *No sample*: A version of OWL-TAMP allowing only one continuous sample per skill.
- *No cont.*: Ablation of OWL-TAMP removing continuous constraint generation (continuous values come purely from TAMP). This is inspired by [9].
- *No disc.*: Ablation of OWL-TAMP removing action-ordering constraints.
- *No backtrack*: Ablation of OWL-TAMP where only the first plan skeleton is refined.

Experimental Setup. We use GPT-4o [4] as the VLM across all methods. Results are averaged over 10 seeds per task, with initial object poses randomized and language goals fixed. CaP-sample uses a 2500-sample budget per task; other methods (except CaP and No-sample, which use 1-sample) have a 500-sample budget per action. Backtracking methods (all OWL-TAMP variants except No-backtrack) may attempt to refine up to 5 plan skeletons. Success is measured by task completion without constraint violation, as judged by manually written detectors; soundness rate (false positive rate) is also reported. Additional metrics (wall-clock time, number of skeletons, and model queries) are in Appendix A.4. All methods use the same standard search-then-sample TAMP system [7, 11, 14, 42], which we detail in Appendix A.1.1.

6.1 Results and Analysis

Table 1 reports success rates. OWL-TAMP achieves the highest success rate on 9 of 10 tasks, outperforming all baselines. Direct trans. succeeds on the 3 simplest tasks, where language goals directly map to existing TAMP predicates, but fails on harder tasks requiring language-conditioned constraint generation. CaP and No sample find correct discrete plans but fail to select continuous parameters satisfying constraints. CaP-sample improves with sampling but struggles when satisfying constraints requires coordinated reasoning across discrete actions. Ablations disabling discrete (No disc.) or continuous (No cont.) constraint generation succeed only on easier tasks, while No backtrack struggles on longer-horizon problems requiring obstacle removal. By generating both discrete and continuous constraints, OWL-TAMP solves all tasks without any task-specific modifications.

Figure 4 reports soundness rates. Direct trans. and No cont. often declare success despite violating the true task intent (a false positive). OWL-TAMP produces almost no false positives — only a single case across all tasks — demonstrating precise grounding of language into constraints. CaP-sample has a high soundness only because it simply fails on most tasks (14% success rate) and declares it fails.

Failures for OWL-TAMP occurred in BerryCook, Mug3, and SoupPour: in BerryCook, the VLM generated an incorrect continuous constraint requiring the strawberry to be inside the bowl and pan simultaneously; in Mug3, the sample budget was exhausted despite correct generated constraints; and in SoupPour, the VLM generated an unsatisfiable plan sketch. By far the most common failure mode was that observed in BerryCook: the system generated infeasible continuous constraints that were impossible to satisfy. While we analyzed OWL-TAMP as a single-shot planner, when deployed in a replanning policy it can use observations from execution or human feedback to augment the language instruction and adapt its behavior after failures.

6.2 Real-World System Deployment

In order to demonstrate our system’s applicability to open-world tasks in the real-world, we deployed OWL-TAMP on a dual-arm robot (Kinova Gen3 arms with a pan-tilt head camera) to perform 19 natural language manipulation tasks, listed below (Figure 2, full list in Appendix A). Please see our supplement for video of the robot’s execution.

- “Put the orange and apple on the plate.”
- “Place the strawberry and lime each in the bin that matches their color.”
- “Stack the blocks into a tower by increasing hue.”
- “Put the apple left of the plate and the orange on the table surface behind of the plate.”
- “Put the orange on the far right of the table and the apple on the far left”.
- “Put the orange where the apple is initially”.
- “Clean the plate” (a sponge is among several other objects present on a tabletop, and the robot must put the sponge atop the plate)
- “Throw away anything not vegan in the purple bin” (objects on the table include a milk carton, apple, spam can, and water bottle).
- “Put the green block between the blue and red ones”
- “Put the blue block onto the plate” (the plate is packed with distractors and the robot must make a tightly-constrained placement).
- “Setup the cutlery for someone to eat a meal from the plate. All the cutlery should be close to and lined-up with the plate, and should be oriented so each is straight and facing forwards, though you should pick which side of the plate each of the items are on” (there are two pieces of fruit, and two similarly-colored blocks that must be disambiguated).
- “Fit one of the fruit in the cup” (only one of 4 available fruits is small enough to fit in the provided mug)
- “Put the brownie ingredients in front of the pan” (only 2 of the available items are related to brownies)
- “Place the cutlery in the utensil holder. All the cutlery should be oriented straight and facing forward”
- “Fry two eggs at the front of the pan”
- “Fry the spam on the pan and serve it on the plate”
- “Weigh the shortest object and put it in the bin”
- “Put the banana near where the other fruit are initially”
- “Place the red block so that it’s aligned with the other two blocks”

In each task, the robot received the camera image and natural language command, estimated the scene geometry using a strategy similar to that in [43], generated constraints and plans via PDDL-Stream [8], and executed plans open-loop on hardware. No changes were made to the system across tasks. OWL-TAMP successfully completed all tasks, despite substantial variation in object configurations and goals. Many tasks required adapting grasps (e.g., side grasps over top-down), managing collisions and reachability, or backtracking to move obstacles. These results highlight OWL-TAMP’s ability to flexibly and robustly integrate vision, language, and planning without any task-specific tuning.

While our system was able to solve versions of each of these tasks, we observed several failures on similar tasks and variants. The primary failure mode of OWL-TAMP was the same as in the simulated experiments: generating incorrect continuous constraints (e.g. the VLM generated a constraint for an object to be in collision with another object). A less common but still prevalent failure mode was generating correct continuous constraints that could not be satisfied with our finite sampling budget (e.g. a particular placement region is unreachable because the real-world robot has a rather limited workspace). The majority of failures of the overall system were caused by common errors made by third-party real-world perception and control modules such as false negatives when segmenting objects and minor collisions due to imperfect trajectory tracking.

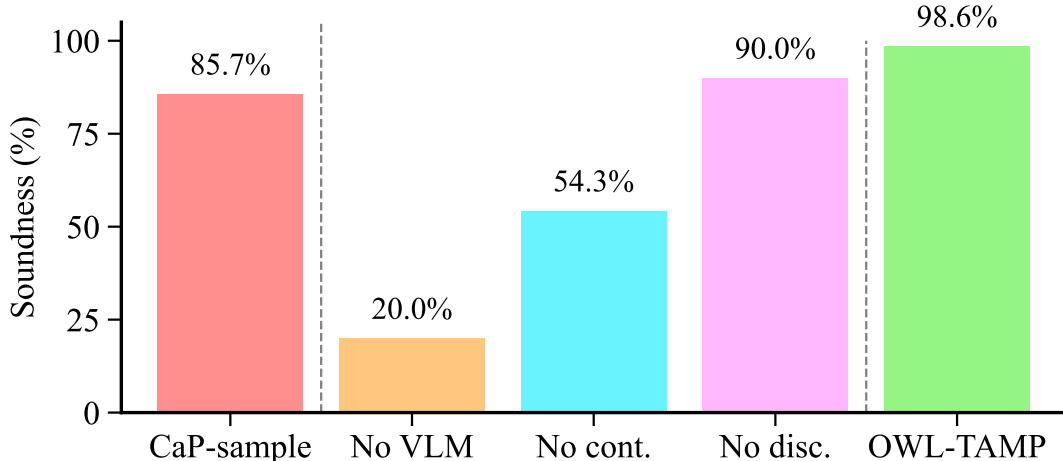


Figure 4: **Soundness rates on 7 tasks.** Average soundness rate ($1 - \frac{\# \text{false positives}}{\# \text{total trials}}$) across tasks requiring non-trivial discrete or continuous constraint generation. A false positive occurs when success is declared despite the output plan violating task intent. $\uparrow$ indicates fewer false positives. Failed tasks (the method correctly declares failure) are counted as true positives.

7 Conclusion

We presented OWL-TAMP, a system that uses VLMs to generate discrete and continuous constraints that enable a base TAMP system to interpret and accomplish open-world language instructions. We demonstrated that OWL-TAMP solves a wide range of complex, long-horizon language-based manipulation tasks more reliably than existing baselines, and that it applies across different robot embodiments in simulation and the real world.

8 Limitations

Our system relies entirely on the VLM to generate constraints that are both syntactically and semantically correct. While the TAMP system is capable of backtracking internally, our current implementation cannot recover from errors in the generated constraints themselves. Another limitation is our assumption of a set of primitive parameterized robot controllers (e.g., moving, rigid grasping, placement) and a library of primitive helper functions — common in planning and foundation model-based code-generation works — which OWL-TAMP uses to generate plans and continuous constraints. Despite these limitations, our results highlight the promise of combining large-scale pretrained models with structured planners, and we hope this work inspires further progress toward flexible, capable robot manipulation systems.

Acknowledgements

Work for this project was partially conducted at NVIDIA Research. We gratefully acknowledge support from NSF grant 2214177; from AFOSR grant FA9550-22-1-0249; from ONR MURI grants N00014-22-1-2740 and N00014-24-1-2603; from ARO grant W911NF-23-1-0034; from the MIT Quest for Intelligence; and from the Boston Dynamics Artificial Intelligence Institute.

References

- [1] W. Huang, C. Wang, Y. Li, R. Zhang, and L. Fei-Fei. Rekep: Spatio-temporal reasoning of relational keypoint constraints for robotic manipulation. In *Conference on Robot Learning (CoRL)*, 2024. URL <https://openreview.net/forum?id=9iG3SEbMnL>.

- [2] A. Curtis, N. Kumar, J. Cao, T. Lozano-Pérez, and L. P. Kaelbling. Trust the PRoc3s: Solving long-horizon robotics problems with LLMs and constraint satisfaction. In *Conference on Robot Learning (CoRL)*, 2024. URL <https://openreview.net/forum?id=r6ZhiVYriY>.
- [3] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng. Code as policies: Language model programs for embodied control. In *arXiv preprint*, 2022. URL <https://arxiv.org/pdf/2209.07753>.
- [4] O. J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altenschmidt, and S. A. et. al. Gpt-4 technical report. In *arxiv preprint*, 2023. URL <https://arxiv.org/pdf/2303.08774>.
- [5] Y. Hu, F. Lin, T. Zhang, L. Yi, and Y. Gao. Look before you leap: Unveiling the power of gpt-4v in robotic vision-language planning, 2023. URL <https://arxiv.org/abs/2311.17842>.
- [6] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0 : A vision-language-action flow model for general robot control, 2024. URL <https://arxiv.org/abs/2410.24164>.
- [7] S. Srivastava, E. Fang, L. Riano, R. Chitnis, S. Russell, and P. Abbeel. Combined task and motion planning through an extensible planner-independent interface layer. In *IEEE international conference on robotics and automation (ICRA)*, 2014. URL <https://people.eecs.berkeley.edu/~russell/papers/icra14-planrob.pdf>.
- [8] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. PDDLStream: Integrating Symbolic Planners and Blackbox Samplers. In *International Conference on Automated Planning and Scheduling (ICAPS)*, 2020. URL <https://arxiv.org/abs/1802.08705>.
- [9] Z. Yang, C. Garrett, D. Fox, T. Lozano-Pérez, and L. P. Kaelbling. Guiding long-horizon task and motion planning with vision language models, 2024. URL <https://arxiv.org/abs/2410.02193>.
- [10] C. R. Garrett, R. Chitnis, R. M. Holladay, B. Kim, T. Silver, L. P. Kaelbling, and T. Lozano-Pérez. Integrated task and motion planning. *CoRR*, abs/2010.01083, 2020. URL <https://arxiv.org/abs/2010.01083>.
- [11] N. Kumar, W. McClinton, K. Le, and T. Silver. Bilevel planning for robots: An illustrated introduction. 2023. URL <https://lis.csail.mit.edu/bilevel-planning-for-robots-an-illustrated-introduction>.
- [12] Y. Xie, C. Yu, T. Zhu, J. Bai, Z. Gong, and H. Soh. Translating natural language to planning goals with large-language models. In *arxiv preprint*, 2023. URL <https://arxiv.org/abs/2302.05128>.
- [13] G. Konidaris, L. P. Kaelbling, and T. Lozano-Pérez. From skills to symbols: Learning symbolic representations for abstract high-level planning. *Journal of Artificial Intelligence Research (JAIR)*, 2018. URL <https://jair.org/index.php/jair/article/view/11175/26380>.
- [14] T. Silver, R. Chitnis, N. Kumar, W. McClinton, T. Lozano-Pérez, L. Kaelbling, and J. B. Tenenbaum. Predicate invention for bilevel planning. In *AAAI Conference on Artificial Intelligence (AAAI)*, 2023. URL <https://ojs.aaai.org/index.php/AAAI/article/view/26429/26201>.
- [15] Y. Liang, N. Kumar, H. Tang, A. Weller, J. B. Tenenbaum, T. Silver, J. F. Henriques, and K. Ellis. Visualpredicator: Learning abstract world models with neuro-symbolic predicates for robot planning. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2410.23156>.

- [16] A. Athalye, N. Kumar, T. Silver, Y. Liang, T. Lozano-Pérez, and L. P. Kaelbling. Predicate invention from pixels via pretrained vision-language models. In *arXiv preprint arXiv:2501.00296*, 2024. URL <https://arxiv.org/pdf/2501.00296>.
- [17] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei. Voxposer: Composable 3d value maps for robotic manipulation with language models. In *Conference on Robot Learning (CoRL)*, 2023. URL <https://voxposer.github.io/voxposer.pdf>.
- [18] H. Huang, F. Lin, Y. Hu, S. Wang, and Y. Gao. Copa: General robotic manipulation through spatial constraints of parts with foundation models. In *arxiv preprint*, 2024. URL <https://arxiv.org/pdf/2403.08248>.
- [19] Y. J. Ma, W. Liang, G. Wang, D.-A. Huang, O. Bastani, D. Jayaraman, Y. Zhu, L. Fan, and A. Anandkumar. Eureka: Human-level reward design via coding large language models, 2024. URL <https://arxiv.org/abs/2310.12931>.
- [20] NVIDIA, :, J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. J. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, J. Jang, Z. Jiang, J. Kautz, K. Kundalia, L. Lao, Z. Li, Z. Lin, K. Lin, G. Liu, E. Llontop, L. Magne, A. Mandlekar, A. Narayan, S. Nasiriany, S. Reed, Y. L. Tan, G. Wang, Z. Wang, J. Wang, Q. Wang, J. Xiang, Y. Xie, Y. Xu, Z. Xu, S. Ye, Z. Yu, A. Zhang, H. Zhang, Y. Zhao, R. Zheng, and Y. Zhu. Gr00t n1: An open foundation model for generalist humanoid robots, 2025. URL <https://arxiv.org/abs/2503.14734>.
- [21] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang, R. Julian, D. Kalashnikov, S. Levine, Y. Lu, C. Parada, K. Rao, P. Sermanet, A. T. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, N. Brown, M. Ahn, O. Cortes, N. Sievers, C. Tan, S. Xu, D. Reyes, J. Rettinghouse, J. Quiambao, P. Pastor, L. Luu, K.-H. Lee, Y. Kuang, S. Jesmonth, N. J. Joshi, K. Jeffrey, R. J. Ruano, J. Hsu, K. Gopalakrishnan, B. David, A. Zeng, and C. K. Fu. Do as i can, not as i say: Grounding language in robotic affordances. In *Conference on Robot Learning (CoRL)*, 2023. URL <https://proceedings.mlr.press/v205/ichter23a.html>.
- [22] W. Huang, F. Xia, D. Shah, D. Driess, A. Zeng, Y. Lu, P. Florence, I. Mordatch, S. Levine, K. Hausman, and B. Ichter. Grounded decoding: Guiding text generation with grounded models for embodied agents. In *arxiv preprint*, 2023. URL <https://arxiv.org/pdf/2303.00855>.
- [23] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg. Progprompt: Generating situated robot task plans using large language models. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. URL <https://arxiv.org/pdf/2209.11302>.
- [24] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter. Inner monologue: Embodied reasoning through planning with language models. In *Conference on Robot Learning (CoRL)*, 2023. URL <https://openreview.net/pdf?id=3R3Pz5i0tye>.
- [25] Y. Guo, Y.-J. Wang, L. Zha, Z. Jiang, and J. Chen. Doremi: Grounding language model by detecting and recovering from plan-execution misalignment. In *arxiv preprint*, 2023. URL <https://arxiv.org/pdf/2307.00329>.
- [26] C. Agia, T. Migimatsu, J. Wu, and J. Bohg. Stap: Sequencing task-agnostic policies. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023. URL <https://arxiv.org/pdf/2210.12250>.
- [27] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. N. Wu, S.-C. Zhu, and H. Liu. Llm³:large language model-based task and motion planning with motion failure reasoning. In *arXiv preprint*, 2024. URL <https://arxiv.org/pdf/2403.11552>.

- [28] W. Yuan, J. Duan, V. Blukis, W. Pumacay, R. Krishna, A. Murali, A. Mousavian, and D. Fox. Robopoint: A vision-language model for spatial affordance prediction in robotics. In *Conference on Robot Learning (CoRL)*, 2024. URL <https://openreview.net/forum?id=GVX6jpZ0hU>.
- [29] J. Duan, W. Yuan, W. Pumacay, Y. R. Wang, K. Ehsani, D. Fox, and R. Krishna. Manipulate-anything: Automating real-world robots using vision-language models. In *Conference on Robot Learning (CoRL)*, 2024. URL <https://openreview.net/forum?id=2SYFDG4WRA>.
- [30] M. Skreta, Z. Zhou, J. L. Yuan, K. Darvish, A. Aspuru-Guzik, and A. Garg. Replan: Robotic replanning with perception and language models. In *arxiv preprint*, 2024. URL <https://arxiv.org/pdf/2401.04157>.
- [31] Y. Ding, X. Zhang, C. Paxton, and S. Zhang. Task and motion planning with large language models for object rearrangement. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023. URL <https://arxiv.org/pdf/2303.06247>.
- [32] Y. Chen, J. Arkin, C. Dawson, Y. Zhang, N. Roy, and C. Fan. Autotamp: Autoregressive task and motion planning with llms as translators and checkers. In *arXiv preprint*, 2024. URL <https://arxiv.org/pdf/2306.06531>.
- [33] S. Kambhampati. Can large language models reason and plan? *Annals of the New York Academy of Sciences*, 2024. URL <https://arxiv.org/pdf/2403.04121>.
- [34] W. Guo, Z. Kingston, and L. E. Kavvaki. Castl: Constraints as specifications through llm translation for long-horizon task and motion planning. In *arxiv preprint*, 2024. URL <https://arxiv.org/abs/2410.22225>.
- [35] D. McDermott, M. Ghallab, A. Howe, C. Knoblock, A. Ram, M. Veloso, D. Weld, and D. Wilkins. PDDL: The Planning Domain Definition Language. Technical report, Yale Center for Computational Vision and Control, 1998. URL <https://www.cs.cmu.edu/~mmv/planning/readings/98aips-PDDL.pdf>.
- [36] B. Bonet and H. Geffner. Planning as heuristic search: New results. In *European Conference on Planning*. Springer, 1999. URL <https://bonetblai.github.io/reports/ecp99-hspr.pdf>.
- [37] J. Hoffmann and B. Nebel. The {FF} Planning System: Fast Plan Generation Through Heuristic Search. *Journal Artificial Intelligence Research (JAIR)*, 14:253–302, 2001. URL <http://dl.acm.org/citation.cfm?id=1622404>.
- [38] S. Edelkamp and M. Helmert. Exhibiting knowledge in planning problems to minimize state encoding length. In *Recent Advances in AI Planning: 5th European Conference on Planning*. Springer, 2000. URL <https://ai.dmi.unibas.ch/papers/edelkamp-helmert-ecp1999.pdf>.
- [39] C. R. Garrett. *Sampling-Based Robot Task and Motion Planning in the Real World*. PhD thesis, Massachusetts Institute of Technology, 2021. URL <https://dspace.mit.edu/bitstream/handle/1721.1/139990/Garrett-cgarrett-PhD-EECS-2021-thesis.pdf?sequence=1&isAllowed=y>.
- [40] A. Zeng, P. Florence, J. Tompson, S. Welker, J. Chien, M. Attarian, T. Armstrong, I. Krasin, D. Duong, V. Sindhwani, and J. Lee. Transporter networks: Rearranging the visual world for robotic manipulation. *Conference on Robot Learning (CoRL)*, 2020. URL <https://proceedings.mlr.press/v155/zeng21a/zeng21a.pdf>.
- [41] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone. Llm+p: Empowering large language models with optimal planning proficiency. *ArXiv*, abs/2304.11477, 2023. URL <https://api.semanticscholar.org/CorpusID:258298051>.

- [42] N. Kumar, W. McClinton, R. Chitnis, T. Silver, T. Lozano-Pérez, and L. P. Kaelbling. Learning efficient abstract planning models that choose what to predict. In *Conference on Robot Learning (CoRL)*, 2023. URL https://openreview.net/pdf?id=_gZLyRGGuo.
- [43] A. Curtis, X. Fang, L. P. Kaelbling, T. Lozano-Pérez, and C. R. Garrett. Long-horizon manipulation of unknown objects via task and motion planning with estimated affordances. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2022. URL <https://arxiv.org/pdf/2108.04145>.
- [44] B. Bonet and H. Geffner. Planning as heuristic search. *Artificial Intelligence*, 2001. URL <https://www.cs.toronto.edu/~sheila/2542/s14/A1/bonetgeffner-heusearch-aij01.pdf>.
- [45] M. Helmert. The Fast Downward Planning System. *Journal of Artificial Intelligence Research (JAIR)*, 26:191–246, 2006. URL <http://www.jair.org/papers/paper1705.html>.
- [46] C. R. Garrett, T. Lozano-Perez, and L. P. Kaelbling. Stripstream: Integrating symbolic planners and blackbox samplers. abs/1802.08705, 2018. URL <https://export.arxiv.org/pdf/1802.08705>.
- [47] C. R. Garrett, T. Lozano-Pérez, and L. P. Kaelbling. Sampling-based methods for factored task and motion planning. *International Journal of Robotics Research (IJRR)*, 2018. URL <https://arxiv.org/pdf/1801.00680>.
- [48] C. R. Garrett, C. Paxton, T. Lozano-Pérez, L. P. Kaelbling, and D. Fox. Online Replanning in Belief Space for Partially Observable Task and Motion Problems. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2020. URL <https://arxiv.org/abs/1911.04577>.
- [49] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, et al. Grounding dino: Marrying dino with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2023. URL <https://arxiv.org/pdf/2303.05499>.
- [50] A. Goyal, J. Xu, Y. Guo, V. Blukis, Y.-W. Chao, and D. Fox. Rvt: Robotic view transformer for 3d object manipulation. In *Conference on Robot Learning (CoRL)*, 2023. URL <https://arxiv.org/pdf/2306.14896>.
- [51] P. Beeson and B. Ames. Trac-ik: An open-source library for improved solving of generic inverse kinematics. In *IEEE-RAS 15th International Conference on Humanoid Robots (Humanoids)*. IEEE, 2015. URL <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7363472>.
- [52] B. Sundaralingam, S. K. S. Hari, A. Fishman, C. Garrett, K. Van Wyk, V. Blukis, A. Millane, H. Oleynikova, A. Handa, F. Ramos, et al. Curobo: Parallelized collision-free robot motion generation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023. URL <https://arxiv.org/pdf/2310.17274>.
- [53] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. V. Le, D. Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html.

A Appendix

We first provide details of how we implement OWL-TAMP (Appendix A.1), including details about its integration with standard TAMP systems, generation of discrete and continuous constraints. Subsequent sections provide more specific details around prompting the VLM (Appendix A.6, A.7, and A.8). For details beyond those presented here, we refer the reader to our open-source code release, which we will make available after acceptance.

A.1 Additional Method Implementation Details

We provide a detailed explanation of how we implement the search-then-sample TAMP system used in our simulation experiments (Section 6.1).

A.1.1 Search-then-Sample TAMP System Details

In our simulation experiments, we implement a relatively simple variant of the search-then-sample TAMP approach used in several recent works [7, 11, 14, 42]. We use this as the underlying TAMP systems for all methods that require one (i.e., all methods except ‘CaP’ and ‘CaP-sample’). We implement versions of the same predicates and operators described in Section 3 (including a version of `transport` that we call `Pour`), though we omit the `move` operator as movement is performed automatically as part of `attach` and `detach` (as illustrated by our prompts in Appendix A.7 below).

We manually define the initial state of all tasks in terms of literals involving these predicates; the robot always begins each task at the same initial configuration, and with the `HandEmpty()` predicate set to `True`. Additionally, we associate each parameterized action with a natural language description to make it easier for the VLM to perform discrete constraint generation (e.g., for the `detach` operator, the description is something like: “places object `o` stably atop a surface; you can specify a description of how this placement should be performed by filling in the `description` parameter accordingly”). Each operator is linked to one particular low-level controller included with the RAVENS-YCB environment [40].

We define an associated sampler for each of these parameterized actions. Each sampler takes in the current state of the task, as well as the action’s discrete arguments, and defines a distribution over the action’s continuous parameters. These samplers are setup to define broad distributions (e.g., the sampler for the `detach` operator simply tries to find a pose somewhere broadly above the surface to be detached onto, but roughly within the confines of the `x` and `y` boundaries of the surface’s axis-aligned bounding box), though we do modify these slightly depending on the task (e.g. the sampler for the ‘Coffee’ and ‘Mug’ tasks defines particular distributions on the orientations the mug and/or cutlery should be detached).

Given these components, we can adopt the following hierarchical search-then-sample planning strategy to achieve a symbolic predicate goal G (derived from a plan sketch via the procedure listed in Appendix A.1.2 from initial state s_0 ²:

1. Compute a new task plan (sequence of ground operators) that achieves G from s_0
2. For b within the backtracking budget:
 - (a) For each ground operator in this plan:
 - i. If the preconditions do not hold, break.
 - ii. For i within the sampling budget:
 - A. Call the associated sampler to get continuous parameters.
 - B. Use these to instantiate the controller associated with the operator.
 - C. Execute the controller within the planning model (i.e., the simulator)
 - D. Check that the resulting state satisfies all the constraints, including manually-engineered and VLM-generated constraints.

²see [11] or [14] for a fuller presentation of the search-then-sample TAMP strategy we employ.

We perform initial task planning via a simple A^* search. During backtracking (i.e., when the sampling budget is exhausted for the first time and a new task plan is required), we employ a set of manually-engineered strategies to modify the task plan based on the most-recent failed operator (e.g. if the most recent-failed operator is a `detach` that was attempting to place an object atop a particular surface, and there are other objects atop that surface already, we randomly append a `attach` sequence to move one of those objects to a different part of the table).

A.1.2 Discrete Constraint Generation Details

Given a goal described in natural language L (Section 3), we first prompt a VLM to generate a *plan sketch* (i.e., a partial plan) that serves as a discrete constraint on the space of TAMP solutions (Section 5.1). To enable this, we associate a natural language description of each available action with that particular action. Although we could directly prompt a VLM for relevant actions and goals, without a list of candidates, the VLM is likely to be syntactically and semantically inaccurate. Instead, we propose to first ground the set of *reachable* actions A and literals Λ (i.e., grounded predicates) available to the TAMP system before prompting the VLM to return values in these sets. We use *relaxed planning* [36, 44] from the initial state s_0 to simultaneously ground [37, 38, 45] and explore the sets of reachable actions A and literals Λ . When instantiating continuous parameters, we use placeholder values, such as *optimistic* values [46, 47, 8, 39], to ensure a finite set of actions are instantiated. Similarly, we use placeholders for description parameters.

Algorithm 2 VLM Task Reasoning

```

1: procedure VLM-TASK-REASONING( $s_0, \mathcal{A}, \mathcal{I}, L$ )
2:    $A \leftarrow \text{GROUND-ACTIONS}(s_0, \mathcal{A})$ 
3:    $\Lambda \leftarrow s_0 \cup \{l \mid a \in A, l \in e.\text{eff}\}$ 
4:    $[a_1, \dots, a_n, l_1, \dots, l_m] \leftarrow \text{QUERY-VLM}(\text{"What partial plan using actions } \{A\} \text{ for goal literals } \{\Lambda\} \text{ achieves goal } \{L\}?" )$ 
5:   for  $i \in [1, n - 1]$  do
6:      $a_i.\text{eff} \leftarrow a_i.\text{eff} \cup \{\text{Executed}(i)\}$ 
7:      $a_{i+1}.\text{pre} \leftarrow a_{i+1}.\text{pre} \cup \{\text{Executed}(i)\}$ 
8:      $a_n.\text{eff} \leftarrow a_n.\text{eff} \cup \{\text{Executed}(n)\}$ 
9:      $G \leftarrow \{l_1, \dots, l_m\}$ 
10:  return SOLVE-TAMP( $s_0, A, G \cup \{\text{Executed}(n)\}$ )

```

Algorithm 2 presents the VLM partial plan generation pseudocode. It takes in a TAMP problem $\langle s_0, \mathcal{A}, \mathcal{I}, L \rangle$, where L is a text goal description and $\mathcal{I}$ is a set of initial image observations of the scene (see Section 3). It first grounds the set of actions A reachable from s_0 using `GROUND-ACTIONS`. Then, it accumulates the set of reachable literals Λ by taking the effects of all actions A . These sets can be filtered by action or predicate type if it is desired to focus VLM assistance on specific aspects of the planning problem. Then, it prompts `QUERY-VLM` for a partial plan $[a_1, \dots, a_n, l_1, \dots, l_m]$ using actions $a_i \in A$ and goal literals $l_j \in \Lambda_m$ that achieve the goal description g . Importantly, we have the VLM fill in the description parameter d for each of these actions. We then transform the original TAMP problem to force solutions to admit the partial plan as a subsequence. Specifically, we create a predicate `EXECUTED` that models whether the i th action in the plan was executed and add `EXECUTED` to the effects of action a_i and the preconditions of action a_{i+1} . Finally, we make the planning goal be $G = \{l_i, \dots, l_m\} \subseteq L$ and `EXECUTED`(n), which indicates that all actions have been executed and solve the transformed TAMP problem with a generic TAMP algorithm.

A.1.3 OWL-TAMP Constraint Generation Example Walkthrough

We walk through a concrete example of generating both discrete and continuous constraints for the ‘Mug1’ task (Figure 3a), where the natural language goal is $L = \text{"Setup the mug so it's upright, then put whatever object that fits inside of it"}$.

First, we prompt the VLM to generate discrete constraints given the task goal. It generates the following partial plan, along with language descriptions filled in for each action:

```

1 ('Pick(mug)', 'grasp the mug securely to lift it from the table.')
2 ('Place_Ontop(mug, table)', 'place the mug upright on the table to ensure it is
   stable.')
3 ('Pick(fork)', 'grasp the fork securely to lift it from the table.')
4 ('Place_Inside(fork, mug)', 'carefully place the fork inside the upright mug.')
5 ('achieve_goal(mug, fork)', 'the goal is achieved when the mug is upright and the
   fork is inside it.')

```

Next, we prompt the VLM to provide continuous constraints for the `achieve_goal` operator at the end of the plan. This corresponds to providing continuous constraints that correspond to the task’s goal description.

```

1 # Goal Check Functions:
2 def goal_check0() -> bool:
3     upright_mug = abs(mug.pose.roll) < 0.1 and abs(mug.pose.pitch) < 0.1 and abs(mug
4     .pose.yaw) < 0.1
5     return upright_mug
6
7 def goal_check1() -> bool:
8     inside_mug_bounds = modify_pose_bounds_to_be_inside_object(init_state, env,
9     init_bounds, mug.category)
10    return position_within_bounds(fork.pose, inside_mug_bounds)

```

We then prompt the VLM to generate continuous constraints for each operator given these generated constraint functions as input. We start from the beginning of the plan. The `Pick(mug)` action (which corresponds to `attach`) does not have any effects or constraint predicates that depend on the VLM, so we skip this action. The `Place_Ontop(mug, table)` action ((which corresponds to `detach`) does feature a `VLMPose` constraint. So we prompt the VLM to generate continuous constraints that implement it based on the language description as well as the goal generated constraints.

```

1 # Goal Check Functions:
2 def goal_check2() -> bool:
3     upright_mug = abs(mug.pose.roll) < 0.1 and abs(mug.pose.pitch) < 0.1 and abs(mug
4     .pose.yaw) < 0.1
5     on_table_bounds = modify_pose_bounds_to_be_ontop_of_object(init_state, env,
6     init_bounds, mug.category, 'table')
7     return upright_mug and position_within_bounds(mug.pose, on_table_bounds)

```

The `Place_Inside(fork, mug)` action (which also corresponds to `detach`) does not have any effects or constraint predicates that depend on the VLM, so we skip this action as well. Finally, the `Place_Inside(fork, mug)` action does have a `VLMPose` constraint, so we once again prompt the VLM, and obtain the following continuous constraints for this action:

```

1 # Goal Check Functions:
2 def goal_check2() -> bool:
3     inside_mug_bounds = modify_pose_bounds_to_be_inside_object(init_state, env,
4     init_bounds, mug.category)
5     return position_within_bounds(fork.pose, inside_mug_bounds)

```

Note that in this particular case, the VLM only checks that the fork is inside the mug, which is a constraint that is built into the TAMP system itself (via the `Inside` predicate that is an effect of the `Place_Inside` action).

Now, we have a partial plan along with continuous constraints for particular actions. We call our underlying TAMP system to attempt to satisfy these, as well as its own internal constraints (Appendix A.1.1).

A.2 Additional RAVENS-YCB Manipulation Experiment Details

We provide additional details on the tasks we ran experiments on in the simulated RAVENS-YCB Manipulation environment (Section 6) and discuss the Code as Policies baseline implementation in Appendix A.2.1.

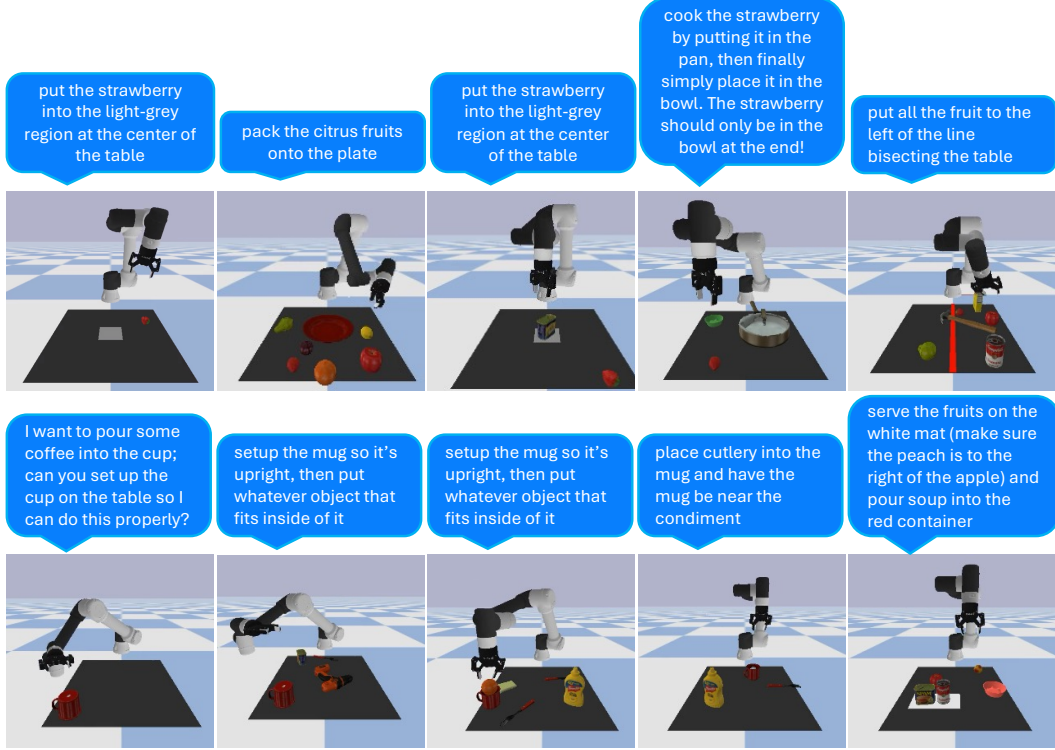


Figure 5: **Ravens YCB-Manipulation tasks visualization.** From top left to bottom right: visualizations of the goal string and an example initial state for the ‘Berry1’, ‘Citrus’, ‘Berry2’, ‘BerryCook’, ‘FruitSort’, ‘Coffee’, ‘Mug1’, ‘Mug2’, ‘Mug3’, and ‘SoupPour’ tasks.

A visualization of an initial state in each of our tasks is shown in Figure 5. In each of these tasks, the robot had access to three parameterized actions: $\text{Pick}(o, g)$ where o is an object and g is a 6-DOF grasp pose consisting of $[x, y, z, \text{roll}, \text{pitch}, \text{yaw}]$ in the world coordinate frame, $\text{Place}(o, s, p)$ where o is the object to be placed, s is the surface or container to place atop or inside, and p is the 6D placement pose at which the robot will move and simply open its gripper, and $\text{Pour}(o, s, t)$, where o is the object to be poured from, s is the surface or container to pour onto or into, and t is a 4D vector consisting of a 3D position and a pitch angle at which to tip the hand to accomplish the pour. For each random seed, the positions of all objects except the ‘table_surface’ object in the scene are randomized unless otherwise stated.

- *Berry1*: Goal description: “put the strawberry onto the light-grey region at the center of the table”. Objects: `strawberry`, `light-grey-region`, `table_surface`. The position of the `light-grey-region` is held constant at the center of the table. The optimal solution to this task requires a sequence of 2 actions.
- *Citrus*: Goal description: “pack the citrus fruit onto the plate”. Objects: `strawberry`, `apple`, `pear`, `lemon`, `orange`, `plum`, `table_surface`. The `plate` object is always set to be at the center of the table in the initial state. The optimal solution to this task requires a sequence of 4 actions.
- *Berry2*: Goal description: “put the strawberry onto the light-grey region at the center of the table”. Objects: `strawberry`, `light-grey-region`, `potted_meat_can`, `table_surface`. The only object whose position is randomized in the initial state is the `strawberry`: the `potted_meat_can` is always set to totally obstruct the surface of the `light-grey-region` in the center of the table: it must be moved out of the way to make room to place the `strawberry` and successfully accomplish the goal description. The optimal solution to this task requires a sequence of 4 actions.

- *BerryCook*: Goal description: “Cook the strawberry by putting it in the pan, then finally simply place it in the bowl. The strawberry should only be in the bowl at the end!”. Objects: strawberry, skillet, bowl, table_surface. The only object whose position is randomized in the initial state is the strawberry. The optimal solution to this task requires a sequence of 4 actions.
- *FruitSort*: Goal description: “Put all the fruit to the left of the line bisecting the table”. Objects: pear, sugar_box, hammer, tomato_soup_can, strawberry, apple, red_line, table_surface. The position of the red_line object is kept constant at the center of the table. The optimal solution to this task requires a sequence of 6 actions.
- *Coffee*: Goal description: “I want to pour some coffee into the cup; can you set up the cup on the table so I can do this properly?” (requires reorienting the cup so that it is placed ‘right-side-up’ on the table). Objects: mug, table_surface. The optimal solution to this task requires a sequence of 2 actions.
- *Mug1*: Goal description: “Setup the mug so it’s upright, then put whatever object that fits inside of it”. Objects: fork, power_drill, potted_meat_can, mug, table_surface. The initial position of the mug is fixed. The optimal solution to this task requires a sequence of 4 actions.
- *Mug2*: Goal description: “Place cutlery inside the mug and then place the mug itself on the table near the condiment” (the mug’s opening is obstructed by a large orange, which must be moved out of the way). Objects: fork, knife, sponge, strawberry, orange, mustard_bottle, mug, table_surface. The initial position of the mug and orange are fixed such that the orange is always obstructing the mouth of the mug. The optimal solution to this task requires a sequence of 8 actions.
- *Mug3*: Goal description: “place cutlery into the mug and have the mug be near the condiment”. This is the same as *CutleryInMug2*, except in the initial state the mug is not obstructed by an orange, but rather has a ball stuck inside it. This must be removed by ‘pouring’ it out of the cup itself). Objects: fork, golf_ball, mustard_bottle, mug, table_surface. The initial position of the mug and golf_ball are fixed such that the golf_ball is always inside the mug. The optimal solution to this task requires a sequence of 8 actions.
- *SoupPour*: Goal description: “Serve the fruits on the white mat (make sure the peach is to the right of the apple” and pour soup into the red container”. The white mat is originally obstructed by the soup can as well as a spam can, which must be moved out of the way to successfully place the fruits. Objects: white_mat, tomato_soup_can, potted_meat_can, bowl, apple, peach, table_surface. The only object positions that are randomized in the initial state are that of the apple and the peach. The optimal solution to this task requires a sequence of 10 actions.

A.2.1 Code as Policies Implementation Details

Following recent work [2], we adapt Code as Policies to our RAVENS YCB-Manipulation domain by having it output a Language Model Program (LMP) function that — given an initial state — directly outputs a full plan with discrete actions and continuous parameters. We provide it with all the same helper functions used for continuous parameter generation that our method has access to. We also provide it with few-shot example solutions to the same 3 example problems as we provide in the continuous constraint generation prompt of our approach. Additionally, we provide it with one sampler per controller that it can use to generate continuous parameters for each respective controller. We provide the full detailed prompt we use for this method below in Appendix A.8.

A.3 Additional Real-World Experiment Details

We attempted the following goal descriptions on the real robot:

- “Put the orange and apple on the plate.”
- “Place the strawberry and lime each in the bin that matches their color.”
- “Stack the blocks into a tower by increasing hue.”
- “Put the apple left of the plate and the orange on the table surface behind of the plate.”

Method	Tasks									
	Berry1	Citrus	Berry2	BerryCook	FruitSort	Coffee	Mug1	Mug2	Mug3	SoupPour
CaP-sample	83.30 ±110.55	2185.30 ±546.48	2353.50 ±363.06	2606.00 ±25.00	0.00 ±0.00	255.60 ±161.55	2519.90 ±3.45	500.90 ±755.41	2557.10 ±9.09	1770.70 ±874.10
No VLM	34.70 ±19.45	247.70 ±81.38	619.40 ±28.15	15.90 ±13.46	0.00 ±0.00	0.00 ±0.00	856.40 ±661.17	819.50 ±374.59	320.20 ±186.27	113.50 ±57.43
No disc.	42.10 ±16.74	303.70 ±85.50	729.30 ±126.84	47.00 ±28.48	0.00 ±0.00	0.00 ±0.00	1636.90 ±349.74	1293.00 ±208.14	915.30 ±244.54	208.70 ±139.25
No cont.	35.10 ±19.37	349.40 ±244.23	193.50 ±115.46	41.00 ±25.54	308.30 ±167.53	132.40 ±78.32	376.80 ±299.85	2500.00 ±0.00	1478.70 ±512.81	1170.60 ±362.64
No back.	37.70 ±16.59	229.10 ±105.03	181.20 ±109.62	36.40 ±17.54	348.20 ±86.87	167.60 ±77.11	348.30 ±145.08	369.50 ±150.42	500.00 ±0.00	450.00 ±113.11
OWL-TAMP	37.70 ±16.59	250.30 ±140.93	181.20 ±109.62	101.90 ±67.60	372.40 ±114.46	167.60 ±77.11	696.30 ±508.87	710.00 ±461.70	1296.30 ±607.64	1335.10 ±409.67
Manual	42.10 ±16.74	303.70 ±85.50	120.60 ±50.45	43.30 ±12.77	271.70 ±99.02	197.80 ±125.70	365.10 ±312.24	657.70 ±666.77	264.40 ±202.57	832.80 ±785.73

Table 2: **Number of samples on all tasks.** We report the average number of continuous parameter sampling attempts for each task across 10 random seeds; the number of samples within the 95% confidence interval is reported after the $\pm$ sign for every entry. We also include a row (‘Manual’) reporting results on a variant of OWL-TAMP where we manually specify ground-truth discrete and continuous constraints to serve as an approximate lower bound on the number of samples our approach could achieve.

- “Put the orange on the far right of the table and the apple on the far left”.
- “Put the orange where the apple is initially”.
- “Clean the plate” (a sponge is among several other objects present on a tabletop, and the robot must put the sponge atop the plate)
- “Throw away anything not vegan in the purple bin” (objects on the table include a milk carton, apple, spam can, and water bottle).
- “Put the green block between the blue and red ones”
- “Put the blue block onto the plate” (the plate is packed with distractors and the robot must make a tightly-constrained placement).
- “Setup the cutlery for someone to eat a meal from the plate. All the cutlery should be close to and lined-up with the plate, and should be oriented so each is straight and facing forwards, though you should pick which side of the plate each of the items are on” (there are two pieces of fruit, and two similarly-colored blocks that must be disambiguated).
- “Fit one of the fruit in the cup” (only one of 4 available fruits is small enough to fit in the provided mug)
- “Put the brownie ingredients in front of the pan” (only 2 of the available items are related to brownies)
- “Place the cutlery in the utensil holder. All the cutlery should be oriented straight and facing forward”
- “Fry two eggs at the front of the pan”
- “Fry the spam on the pan and serve it on the plate”
- “Weigh the shortest object and put it in the bin”
- “Put the banana near the other fruit”
- “Place the red block so that it’s aligned with the other two blocks”

Please see our supplementary video to see the real robot execution.

A.4 Additional Experimental Results

In this section, we explore experimental results related to how much computation and time our various methods took on each of the simulated tasks in the RAVENS YCB-Manipulation domain from Section 6. We also present a fine-grained table of soundness rate results used to construct Figure 4 in Table 4.

Table 2 shows the number of samples required by the various methods to solve each of the RAVENS-YCB Manipulation tasks. Note importantly that we do not adjust for success rates (depicted in Table 1) here: many approaches that achieve a low number of samples (e.g. CaP-sample or No VLM on FruitSort) here actually do so only because they in fact fail to translate that particular task into a form that they can attempt to solve via sampling. OWL-TAMP generally does not require significantly more samples than the ‘Manual’ baseline (which serves as an approximate lower bound on the samples). In two of the tasks where it does require substantially more samples than ‘Manual’

Method	Tasks									
	Berry1	Citrus	Berry2	BerryCook	FruitSort	Coffee	Mug1	Mug2	Mug3	SoupPour
CaP	10.03	21.27	12.63	13.77	19.50	9.66	19.77	23.25	16.37	19.33
	± 0.32 (47.38%)	± 1.10 (25.02%)	± 1.13 (42.76%)	± 0.95 (45.88%)	± 0.67 (26.14%)	± 0.80 (47.61%)	± 2.84 (46.16%)	± 1.65 (30.42%)	± 1.49 (38.14%)	± 1.17 (42.88%)
CaP-sample	22.40	248.49	245.31	262.43	20.88	40.39	281.74	75.61	274.50	187.90
	± 12.70 (21.22%)	± 54.38 (2.14%)	± 35.87 (2.20%)	± 14.12 (2.41%)	± 2.05 (24.41%)	± 16.21 (11.39%)	± 2.99 (3.24%)	± 78.73 (9.35%)	± 11.97 (2.27%)	± 83.47 (4.41%)
No VLM	11.90	41.59	47.66	15.96	16.56	4.92	59.43	78.72	36.03	21.28
	± 1.35 (8.85%)	± 4.85 (4.53%)	± 3.20 (1.60%)	± 1.23 (29.77%)	± 0.14 (24.60%)	± 0.04 (10.25%)	± 31.91 (4.11%)	± 18.85 (6.36%)	± 11.85 (8.04%)	± 2.54 (20.83%)
No sample	44.00	75.17	59.47	54.93	81.19	39.95	55.07	80.00	46.22	91.46
	± 4.03 (85.97%)	± 5.26 (72.29%)	± 4.69 (83.77%)	± 4.68 (82.13%)	± 10.40 (74.17%)	± 4.28 (85.49%)	± 3.43 (73.59%)	± 9.63 (69.38%)	± 2.57 (70.58%)	± 29.02 (82.12%)
No disc.	44.92	91.62	80.87	44.50	36.10	18.92	118.42	120.94	81.85	96.95
	± 4.07 (70.03%)	± 6.22 (50.92%)	± 6.83 (33.37%)	± 4.38 (69.50%)	± 2.01 (59.87%)	± 2.96 (72.29%)	± 16.71 (20.71%)	± 9.99 (29.63%)	± 10.58 (26.85%)	± 6.37 (72.54%)
No cont.	18.15	54.53	36.91	24.30	57.03	25.73	52.26	136.88	102.53	91.00
	± 1.84 (40.92%)	± 13.09 (16.21%)	± 5.61 (29.48%)	± 1.91 (29.40%)	± 9.88 (15.68%)	± 4.13 (29.93%)	± 16.08 (14.61%)	± 5.72 (5.72%)	± 22.20 (6.04%)	± 20.21 (9.42%)
No back.	48.69	106.36	101.84	67.94	149.53	57.68	87.31	106.82	69.59	111.50
	± 3.40 (71.92%)	± 24.85 (60.35%)	± 34.28 (72.59%)	± 15.09 (75.11%)	± 39.78 (66.52%)	± 8.20 (65.30%)	± 21.78 (58.23%)	± 6.13 (54.49%)	± 3.02 (52.24%)	± 32.13 (67.36%)
OWL-TAMP	46.92	108.50	101.88	71.38	154.32	57.96	113.18	140.51	118.38	173.81
	± 3.37 (74.64%)	± 24.61 (59.15%)	± 34.47 (72.56%)	± 15.31 (71.49%)	± 43.43 (64.46%)	± 7.69 (64.98%)	± 28.50 (44.92%)	± 23.06 (41.43%)	± 23.74 (30.71%)	± 49.00 (43.21%)
Manual	13.53	44.42	24.84	19.78	49.72	22.15	45.18	79.91	37.64	81.38
	± 2.36 (0.00%)	± 5.34 (0.00%)	± 2.78 (0.00%)	± 1.34 (0.00%)	± 3.59 (0.00%)	± 6.36 (0.00%)	± 16.74 (0.00%)	± 33.04 (0.00%)	± 10.40 (0.00%)	± 41.85 (0.00%)

Table 3: **Wall clock time on all tasks.** We report the average wall clock time in seconds each approach took to solve each task across 10 random seeds; the time within the 95% confidence interval is reported after the $\pm$ sign, and the average percentage of that time that was spent querying a foundation model is reported within parentheses ‘()’ for every entry. We also include a row (‘Manual’) reporting results on a variant of OWL-TAMP where we manually specify ground-truth discrete and continuous constraints (and thus no need to query a foundation model for these) to serve as an approximate lower bound on the time our approach could achieve.

(Mug3, SoupPour), OWL-TAMP provides discrete constraints that yield an initial plan skeleton that is incorrect, necessitating backtracking within the TAMP system, which significantly increases the number of samples.

Table 3 shows the wall clock time required by the various methods to solve each of the RAVENS-YCB Manipulation tasks. Note once again that we do not adjust for success rates here. In general, we see that our approach takes significantly more wall clock time than the ‘Manual’ oracle, but we also see that a substantial portion of the time taken (around 50% on average) is due to querying a foundation model. We also see that the percentage of time spent querying increases roughly with the task complexity (left to right), which is what we would expect, since in the more complex tasks with more constraints, it is more challenging to find a satisfying sample. These findings indicate that the overall runtime of our method could be significantly reduced by reducing the time taken for foundation model querying.

A.5 Real Robot System Implementation

For the real-world demos, we adapted the strategy of Curtis et al. [43], which deploys TAMP without a priori object models by estimating collision, grasp, and placement affordances online. Here, we used PDDLStream and its ‘Adaptive’ solver [8] to implement the TAMP framework and algorithm within OWL-TAMP, demonstrating OWL-TAMP’s compatibility with multiple off-the-shelf TAMP systems. We deployed OWL-TAMP in a replanning policy [48], where at each state, the robot observes the world with its head camera, segments and estimates the object geometries, plans a course of action, and executes its plan. We used Grounding DINO [49] for object detection, segmentation,

Method	Tasks						
	BerryCook	FruitSort	Coffee	Mug1	Mug2	Mug3	SoupPour
CaP-sample	100%	100%	0%	100%	100%	100%	100%
No VLM	0%	0%	0%	20%	0%	20%	100%
No disc.	40%	100%	90%	100%	100%	100%	100%
No cont.	100%	10%	60%	70%	100%	20%	20%
OWL-TAMP	100%	100%	100%	100%	100%	100%	90%

Table 4: **Soundness rates on select methods and tasks.** For tasks that require non-trivial discrete or continuous constraint generation, we present the ‘soundness rate’ (i.e., $1 - \frac{\# \text{ false positives}}{\# \text{ total trials}}$), where higher rates indicate that the method has fewer false positives.

and association, RVT [50] to infer grasp affordances, TRAC-IK [51] for inverse kinematics, and cuRobo [52] for motion planning.

A.6 Helper Functions for Continuous Constraint Generation.

We provide all methods (Section 6) access to the following helper functions to be used towards constraining the continuous parameters used to instantiate actions. We provide the name, parameters and docstring of each function exactly as below:

```

1 def get_aabb_bounds
2     """Given the state of a particular env, and an object_name that appears in
3     this state, return tuples corresponding to the bounds of the axis-aligned
4     bounding box of object_name in this state in the world frame.
5
6     In particular, return the lower and upper bounds on the axis-aligned
7     x, y, z values.
8     """
9
10 def get_obj_center
11     """Given the state of a particular env, return the pose of the object with
12     object_name.
13
14     The pose is a tuple of dim 6 corresponding to (x, y, z, roll, pitch,
15     yaw).
16     """
17
18 def modify_pose_bounds_to_be_behind_object
19     """Given a tuple of initial bounds (init_bounds), return a modified set of
20     bounds such that sampling randomly from the output bounds will ensure that
21     a pose will be selected that is behind (on the table plane) the object with
22     name 'object_name's. For instance:
23     modify_pose_bounds_to_be_behind_object(init_state, env, init_bounds,
24     'hammer') will modify init_bounds such that they only contain poses that
25     are behind the 'hammer' object on the table surface ahead the robot.
26
27     Note that this does not constrain
28     the pose's horizontal position (it may be anywhere on the table - in the
29     left or right half - such that it's behind object_name).
30     """
31
32 def modify_pose_bounds_to_be_in_front_of_object
33     """Given a tuple of initial bounds (init_bounds), return a modified set of
34     bounds such that sampling randomly from the output bounds will ensure that
35     a pose will be selected that is to the in front of (on the table plane) the
36     object with name 'object_name's. For instance:
37     modify_pose_bounds_to_be_in_front_of_object(init_state, env, init_bounds,
38     'hammer') will modify init_bounds such that they only contain poses that
39     are in front of the 'hammer' object on the table surface ahead the robot.

```

```

40
41     Note that this does not constrain
42     the pose's horizontal position (it may be anywhere on the table - in the
43     left or right half - such that it's in front of object_name).
44     """
45
46 def modify_pose_bounds_to_be_left_of_object
47     """Given a tuple of initial bounds (init_bounds), return a modified set of
48     bounds such that sampling randomly from the output bounds will ensure that
49     a pose will be selected that is to the left of (on the table plane) the
50     object with name 'object_name's. For instance:
51     modify_pose_bounds_to_be_left_of_object(init_state, env, init_bounds,
52     'hammer') will modify init_bounds such that they only contain poses that
53     are to the left of the 'hammer' object on the table surface ahead the
54     robot.
55
56     Note that this does not constrain
57     the pose's vertical position (it may be anywhere on the table - in the
58     upper or lower half - such that it's to the left of object_name).
59     """
60
61 def modify_pose_bounds_to_be_right_of_object
62     """Given a tuple of initial bounds (init_bounds), return a modified set of
63     bounds such that sampling randomly from the output bounds will ensure that
64     a pose will be selected that is to the right of (on the table plane) the
65     object with name 'object_name's. For instance:
66     modify_pose_bounds_to_be_right_of_object(init_state, env, init_bounds,
67     'hammer') will modify init_bounds such that they only contain poses that
68     are to the right of the 'hammer' object on the table surface ahead the
69     robot.
70
71     Note that this does not constrain
72     the pose's vertical position (it may be anywhere on the table - in the
73     upper or lower half - such that it's to the right of object_name).
74     """
75
76 def modify_pose_bounds_to_be_above_object
77     """Given a tuple of initial bounds (init_bounds), return a modified set of
78     bounds such that sampling randomly from the output bounds will ensure that
79     a pose will be selected that is above (on the table plane) the object with
80     name 'object_name's. For instance:
81     modify_pose_bounds_to_be_above_object(init_state, env, init_bounds,
82     'hammer') will modify init_bounds such that they only contain poses that
83     are above the 'hammer' object on the table surface ahead the robot.
84
85     Note that this does actually also constrain the pose's horizontal
86     position and vertical positions so that it is directly above the
87     object in question. Note also that this function might particularly
88     useful to constraint pouring actions (because pouring must be done
89     from above); though you will also likely have to apply an additional
90     angular constraint (since this function doesn't apply any angular
91     constraints on its own).
92     """
93
94 def modify_pose_bounds_to_be_below_object
95     """Given a tuple of initial bounds (init_bounds), return a modified set of
96     bounds such that sampling randomly from the output bounds will ensure that
97     a pose will be selected that is below (on the table plane) the object with
98     name 'object_name's. For instance:
99     modify_pose_bounds_to_be_below_object(init_state, env, init_bounds,
100     'hammer') will modify init_bounds such that they only contain poses that
101     are below the 'hammer' object on the table surface ahead the robot.
102
103     Note that this does actually also constrain the pose's horizontal
104     position and vertical positions so that it is directly below the

```

```

105     object in question. Note also that this function might particularly
106     useful to constraint pouring actions (because pouring must be done
107     from above into a container that's below); though you will also
108     likely have to apply an additional angular constraint (since this
109     function doesn't apply any angular constraints on its own).
110     """
111
112     def modify_pose_bounds_to_be_near_object
113         """Given a tuple of initial bounds (init_bounds), return a modified set of
114         bounds such that sampling randomly from the output bounds will ensure that
115         a pose will be selected such that the distance of the pose from the object
116         with name 'object_name' will be within closeness_thresh along the x, y, and
117         z axes respectively (note that the pose might have an L2) distance that's
118         greater than that."""
119
120     def modify_pose_bounds_to_be_ontop_of_object
121         """Assuming the init_bounds are on the pose (x, y, z, roll, pitch, yaw) of
122         an object with name obj1_name, modify these such that the pose must be
123         confined to be on top of the object with name obj2_name.
124
125         Specifically, restrict the bounds to be within x and y of
126         obj2_name's bounding-box, but have its z-position touching the top
127         of the bounding box of obj2_name.
128
129         IMPORTANT: use this only when trying to place an object atop another
130         (e.g. atop a region, or a surface of another object). If you want to put
131         something inside a container, use the
132         modify_pose_bounds_to_be_ontop_of_object function instead.
133         """
134
135     def modify_pose_bounds_to_be_inside_object
136         """Assuming the init_bounds are on the pose (x, y, z, roll, pitch, yaw) of
137         an object with name obj1_name, modify these such that the pose must be
138         confined to be inside the object with name obj2_name.
139
140         Specifically, restrict the bounds to be within x and y of
141         obj2_name's bounding-box.
142
143         IMPORTANT: use this only when trying to place an object inside a container
144         (e.g. a cup, or vase, or 3D box). If you want to put something in a 2D
145         region, use the modify_pose_bounds_to_be_ontop_of_object function instead.
146         Also note that this function is generally not suitable to constrain
147         pouring; it should generally be used when constraining placement!
148         """
149
150     def position_within_bounds
151         """Checks that the xyz position component of a 6-d pose is within specific
152         bounds."""
153
154     def initialize_bounds_anywhere_on_object
155         """Given obj, get its aabb and initialize bounds such that sampling within
156         these bounds will yield a pose with a position atop obj and any arbitrary
157         rotation."""
158
159     def sample_ravenpose_uniformly_within_bounds
160         """Given obj, get its aabb and initialize bounds such that sampling within
161         these bounds will yield a pose with a position atop obj and any arbitrary
162         rotation."""
163
164     def modify_obj_pose
165         """Modifies the pose of obj to new_pose."""

```

A.7 OWL-TAMP prompting details.

As described in Section 5, our approach consists of an initial discrete constraint generation phase followed by continuous constraint generation based on a discrete partial plan skeleton with language parameters filled in (i.e., plan sketch).

We use separate prompts for both the discrete and continuous constraint generation. For both discrete and continuous constraint generation, we provide an image of the initial state of the task as part of the prompt.

Our discrete constraint generation prompt provides a single few-shot example of expected output on a particular task, and then requests the VLM to output a plan in a similar format for the current task. It also leverages chain of thought prompting [53] to encourage the model to improve the output accuracy. The full prompt we use is shown below: variables within curly brackets ({}) are filled-in dynamically depending on the task.

```
You are an expert-level robot task planning system whose job is to help a robot
accomplish the following task: ''{task_str}''.

Here is the initial predicate state (i.e., the set of all ground atoms that are true
) of this task. Note that an image corresponding to the environment in this
state
is attached below:
{initial_preds}

Your job is to output a sequence of ground operators (i.e., a plan) that ideally
achieve the goal from this initial state.
Your plan need not be perfect, but it should capture the critical objects and
actions necessary to accomplish this task (e.g.
if the task requires 4 objects being in a specific location, then you should take
care to make sure the plan contains
an action to manipulate each of the 4 objects in turn).

Here are the unground operators with their descriptions.
{nsrts_description}

Here are all the ground operators available to you; each operator you use in your
plan must be one of these.
{ground_operators}

Along with each operator in your plan, you should also output a natural language
description of what that operator should
do. This description can be as detailed as you like, and should explain any details
relevant to completing the particular
ground operator successfully.

As an example, consider the example task ''serve the banana inside the blue thing''.
Here, the bowl happens to be blue, and
the initial state is:
OnTable(banana)
OnTable(bowl)
And the available ground operators are:
pick(banana)
pick(bowl)
pick(table)
place_ontop(banana, bowl)
place_inside(banana, bowl)
place_ontop(bowl, banana)
place_inside(bowl, banana)
place_ontop(banana, table)
place_inside(banana, table)
place_ontop(bowl, table)
place_inside(bowl, table)
place_ontop(table, bowl)
```

```
place_inside(table, bowl)
```

Given this, the output should be something like:

```
"""
In the initial state, there is a blue bowl on the table, and a banana atop the table
. The banana is not in the bowl, and the task is to
move the banana into the bowl.
The main actions relevant to the task are 'pick(banana)' and 'place(banana, bowl)'.
The goal involves a relationship between the banana and the bowl only.
All other objects can be ignored.
Plan:
pick(banana); make a stable grasp on the banana - try to make a top-down grasp for
maximum likelihood of success
place(banana, bowl); place the banana stably so that it rests in the bowl - the
banana is too large to fit inside the bowl if it is placed flatly: it needs to
be reoriented to be upright so that it can fit into the bowl
achieve_goal(banana, bowl); the goal involves the banana being inside the bowl -
this relationship is purely between the banana and bowl and doesn't involve/
require any other objects.
"""
```

Notice how the plan ends in an 'achieve_goal' operator. Every plan you output should end with such an operator, and the object arguments to this operator (i.e., '(banana, bowl)' in this case) should be all the objects necessary to decide whether or not the goal has been achieved (i.e., do your best not to include extraneous objects that are irrelevant to deciding whether the task goal has been achieved or not).

Please output your plan in the following format (do not include the angle brackets: those are just for illustrative purposes). Importantly, please do not list the plan with a numbered or bulleted list, simply output each ground operator on a new line with no marking in front of the line as indicated below.

```
<description of the initial state and task in your own words>
<description of which objects and actions are particularly relevant to solving the
task>
<description of any challenges or other important considerations/obstacles that
might arise when solving the task>
Plan:
<ground_operator0>; <natural language description0>
<ground_operator1>; <natural language description1>
...
<ground_operatorm>; <natural language descriptionm>
```

Note that we ask the model to output a `achieve_goal` operator at the end of the plan. This is used by the continuous constraint generation procedure that follows: we generate continuous constraints for this action (the natural language description of the `achieve_goal` action is effectively the task's goal description *g*) first — which effectively corresponds to generating constraints for the task goal — and then generate constraints for any previous operators in the plan by conditioning on these goal continuous constraints.

We prompt the VLM to generate goal continuous constraints by providing the helper functions listed above, available objects in the scene, as well as three few-shot examples (shown below) of outputs on three separate simple example problems:

To give you an idea of what your output function should look like, here is an example function generated for the task "put the lemon on the spoon and the banana on the table", where "lemon", "spoon", "banana", and "table" are all objects in that task/scene.

```
'''python
def goal_check0() -> bool:
    onto_spoon_bounds = modify_pose_bounds_to_be_ontop_of_object(init_state, env,
        init_bounds, lemon.category, spoon.category)
```

```

    return position_within_bounds(lemon.pose, ontop_spoon_bounds)
'''
'''python
def goal_check1() -> bool:
    ontop_table_bounds = modify_pose_bounds_to_ be_ontop_of_object(init_state, env,
        init_bounds, banana.category, table.category)
    return position_within_bounds(banana.pose, ontop_table_bounds)
'''

```

Here is another example set of functions generated for the task "serve the banana inside the blue thing after drying it by placing on the plate". The initial state for this example

is shown in one of the attached images. Here, 'banana' and 'bowl' are both objects (the bowl happens to be blue, whereas the plate is red).

The initial state in this example is:

```

bowl: Pose=RavenPose(x=-0.09269248694181442, y=-0.7042829990386963, z
    =0.026169249787926674, roll=0.0, pitch=-0.0, yaw=0.8605557025412023)
banana: Pose=RavenPose(x=0.17416073374449514, y=-0.33348321026557554, z
    =0.02017684663429707, roll=5.081222700168695e-05, pitch=0.00013538346655467005,
    yaw=-3.0371082921616765)
plate: Pose=RavenPose(x=-0.11636300384998322, y=-0.4429782032966614, z
    =0.014744692512349077, roll=7.884650441866775e-28, pitch=-7.554679105908491
    e-28, yaw=2.245637386214381)
table: Pose=RavenPose(x=0.0, y=-0.5, z=0.0, roll=0.0, pitch=-0.0, yaw=0.0)

```

Importantly, notice how the 'goal_check' function checks that the banana is 'upright' in the bowl by checking its rotation is 90 degrees (approx. 1.57 radians) along the roll axis. This is necessary, because the banana only fits into the bowl in this orientation, as shown in another example rendered image attached below. Pay careful attention and think about any similar orientation constraints that might be necessary in new problems.

```

'''python
def goal_check0() -> bool:
    in_bowl_bounds = modify_pose_bounds_to_ be_inside_object(init_state, env,
        init_bounds, bowl.category)
    banana_in_bowl_bounds = position_within_bounds(banana.pose, in_bowl_bounds)
    is_upright = 1.4 <= abs(banana.pose.roll) <= 1.65
    return banana_in_bowl_bounds and is_upright
'''

```

Notice here that only one 'goal_check' function was each defined, because satisfying the goal depends on all the continuous variables jointly.

Notice also that the 'goal_check' doesn't test for anything to do with the plate, even though "drying" the banana in the plate was important to the task. This is because - in the final state - the banana should be in the bowl (it should have previously been placed in the cup), and the 'goal_check' function only operates in the final state.

Finally, here's an example of constraints for a task "serve spam from its can into the cup". Here, the objects available are 'potted_meat_can' and 'mug'.

```

'''python
def goal_check0() -> bool:
    above_mug_bounds = modify_pose_bounds_to_ be_above_object(init_state, env,
        init_bounds, mug.category)
    above_mug = position_within_ bounds(potted_meat_can.pose, above_bowl_bounds)
    pour_angle_sufficient = abs(potted_meat_can.pose.roll) > 1.2
    return above_bowl and pour_angle_sufficient
'''

```

Notice once again that only one 'goal_check' function was defined.

Notice also that the function checks the roll of the 'potted_meat_can', because this is important to know that it has been sufficiently 'tipped-over' such that its contents can fall from the bowl inside it into the cup.

Carefully consider these examples to inform your own functions for the current problem.

...

We use these same few-shot examples across all tasks without varying them. We prompt the model to output `goal_check` functions for the current goal and extract these. Then, for each of the previous actions in the plan with language descriptions as well as predicates that rely on the VLM for their implementation, we prompt the VLM for continuous constraints that should hold after that particular action is executed.

A.8 Code as Policies Full Prompt

Our full prompt for Code as Policies is shown below. Note that variables within curly brackets (`{}`) are filled-in dynamically depending on the task.

```
You are operating in an environment that has access to the following classes: {
    env_structs}.

You have access to the following variables. Note that TABLE_AABB.lower and
    TABLE_AABB.upper are tuples of length 3 that represent the lower and upper
    bounds on the x, y, and z positions that are on the table.
{typed_variables}

You also have access to the following objects in the environment.
Each of these has type 'RavenObject'. You are not able to use other objects:
{object_names}

Note that when you use/invoke these objects, do not use them as a string (i.e., do
    not provide quotes ''). Instead
    use them directly (e.g. use banana, not 'banana'). Use the name of the object as a
    string only where explicitly required
    (e.g. by the GraspSampler).

Here's the poses of all the objects in the initial state of the scene (depicted in
    an attached image). Note that the axes of each object is shown as red (x-axis),
    green (y-axis), and blue (z-axis).
Note that roll is rotation about the x-axis, pitch is rotation about the y-axis and
    yaw is rotation about the z-axis. Pay careful attention to the axes and the
    current orientations of objects in the initial state as and when you decide to
    write functions involving orientations.
{init_state}

You also have access to helper functions whose signatures and docstrings are shown
    below. Pay careful attention to the arguments and return values of each
    function.
Helper function signatures:
{helper_functions_and_docstrings}

You have access to the following set of skills expressed as pddl predicates followed
    by descriptions.
You have no other skills you can use, and you must exactly follow the number of
    inputs described below.
The coordinate axes are x, y, z where x is distance from the robot base, y is left/
    right from the robot base, and z is the height off the table.

Action("pick", [[o], g])
Pick up object o at grasp g sampled from a grasp sampler. Note that you should use
    the object o directly and not use its name (e.g. banana and not 'banana').

Action("place_ontop", [[o, s], p])
If holding an object o (e.g. 'banana') place the object ontop surface s (e.g. 'table
    ') at pose p. Note that you should invoke s and o as objects directly and not
    use their names (e.g. banana and not 'banana').
```

Action("place_inside", [[o, c], p])
 If holding an object o (e.g. 'banana') place the object inside container c (e.g. 'bowl') at placement pose p. Note that you should invoke o and c as objects directly and not use their names (e.g. banana and not 'banana').

Action("pour", [[o, c], p])
 If holding an object 'o' (e.g. 'tomato_soup_can') pour from the object onto/inside of container 'c' (e.g. 'bowl') at a pouring pose dictated by p. Specifically, p is 4 numbers: <x, y, z> corresponding to the position to reach, and <theta> corresponding to the angle (in radians; -3.14 to 3.14) to tilt the hand to accomplish the pour. Note that you should invoke o and c as objects directly and not use their names (e.g. banana and not 'banana').

Finally, to help you generate continuous parameters required for these skills, you have access to the following samplers.

```
@dataclass
class GraspSampler(Sampler):
    curr_state: RavenState
    env: RavenYCBEnv
    object_name: str

    def sample(self, rng: np.random.Generator) -> List[float]:
        """Simply return some value within the AABB of the object, and at any
        orientation."""
        aabb_lb, aabb_ub = get_aabb_bounds(
            self.curr_state, self.env, self.object_name
        )
        lb = (aabb_lb[0], aabb_lb[1], aabb_lb[2], -3.14159265, -3.14159265, -3.
            14159265)
        ub = (aabb_ub[0], aabb_ub[1], aabb_ub[2], 3.14159265, 3.14159265,
            3.14159265)
        return rng.uniform(lb, ub).tolist()

@dataclass
class PlaceSampler(Sampler):
    curr_state: RavenState
    env: RavenYCBEnv
    object_name_to_place_on_or_inside: str

    def sample(self, rng: np.random.Generator) -> List[float]:
        """Simply return some value within the AABB of the object, and at any
        orientation."""
        min_drop_height = 0.01
        max_drop_height = 0.35
        aabb_lb, aabb_ub = get_aabb_bounds(
            self.curr_state, self.env,
            self.object_name_to_place_on_or_inside
        )
        lb = (
            aabb_lb[0],
            aabb_lb[1],
            aabb_lb[2] + min_drop_height,
            -3.14159265,
            -3.14159265,
            -3.14159265,
        )
        ub = (
            aabb_ub[0],
            aabb_ub[1],
            aabb_ub[2] + max_drop_height,
            3.14159265,
            3.14159265,
            3.14159265,
        )
```

```

    )
    return rng.uniform(lb, ub).tolist()

@dataclass
class PourSampler(Sampler):
    curr_state: RavenState
    env: RavenYCBEnv
    holding_obj_name: str
    obj_name_to_pour_into_or_ontop: str

    def sample(self, rng: np.random.Generator) -> List[float]:
        """Return a position sampled from above obj_name_to_pour_into_or_ontop
        and a pitch randomly sampled between bounds."""
        # We sample (x, y, z, pitch.)
        obj_holding_lb, obj_holding_ub = get_aabb_bounds(
            self.curr_state, self.env, self.holding_obj_name
        )
        min_pour_height = obj_holding_ub[2] - obj_holding_lb[2]
        max_pour_height = min_pour_height * 2
        aabb_lb, aabb_ub = get_aabb_bounds(
            self.curr_state, self.env, self.obj_name_to_pour_into_or_ontop
        )
        pitch_range = (-3.14, 3.14)
        lb = (aabb_lb[0], aabb_lb[1], aabb_lb[2] + min_pour_height, pitch_range[0])
        ub = (aabb_ub[0], aabb_ub[1], aabb_ub[2] + max_pour_height, pitch_range[1])
        return rng.uniform(lb, ub).tolist()

```

Your goal is to generate a python function that returns a plan that performs the provided task. This function can use helper functions that must be defined within the scope of the function itself.

The main function should be named EXACTLY 'gen_plan', and it should take in only one parameter corresponding to the environment state as input. Do not change the names. Do not create any additional classes or overwrite any existing ones. You are only allowed to create helper functions inside the 'gen_plan' function.

Current provided goal: {provided_task_goal}

Here is an example 'gen_plan' function for a different task:

```

#define user
Init state:
bowl: Pose=RavenPose(x=-0.09269248694181442, y=-0.7042829990386963, z
=0.026169249787926674, roll=0.0, pitch=-0.0, yaw=0.8605557025412023)
banana: Pose=RavenPose(x=0.17416073374449514, y=-0.33348321026557554, z
=0.02017684663429707, roll=5.081222700168695e-05, pitch=0.00013538346655467005,
yaw=-3.0371082921616765)
plate: Pose=RavenPose(x=-0.11636300384998322, y=-0.4429782032966614, z
=0.014744692512349077, roll=7.884650441866775e-28, pitch=-7.554679105908491
e-28, yaw=2.245637386214381)
table: Pose=RavenPose(x=0.0, y=-0.5, z=0.0, roll=0.0, pitch=-0.0, yaw=0.0)
Task goal: put the lemon on the plate and the banana on the table

#define assistant
'''python
def gen_plan(initial:RavenState):
    plan = []
    place_pose = PlaceSampler(initial, env, "plate").sample(rng)
    lemon_grasp = GraspSampler(initial, env, "lemon").sample(rng)
    plan.append(Action("pick", [[lemon], lemon_grasp]))
    plan.append(Action("place_ontop", [[lemon, plate], place_pose]))
    place_pose = PlaceSampler(initial, env, "table").sample(rng)
    banana_grasp = GraspSampler(initial, env, "banana").sample(rng)
    plan.append(Action("pick", [[banana], banana_grasp]))

```

```

    plan.append(Action("place_ontop", [[banana, table], RavenPose(x=x_place, y=
        y_place, z=0.02)]))
    return plan
'''

```

Here is another example for a different goal involving the same objects and initial state. Note that in this case, the bowl is blue.

Also note that the banana needs to be rotated so that it fits into the bowl; in general you should pay careful attention to any angular constraints that might be important for solving different tasks.

```

#define user
Init state:
bowl: Pose=RavenPose(x=-0.09269248694181442, y=-0.7042829990386963, z
    =0.026169249787926674, roll=0.0, pitch=-0.0, yaw=0.8605557025412023)
banana: Pose=RavenPose(x=0.17416073374449514, y=-0.33348321026557554, z
    =0.02017684663429707, roll=5.081222700168695e-05, pitch=0.00013538346655467005,
    yaw=-3.0371082921616765)
plate: Pose=RavenPose(x=-0.11636300384998322, y=-0.4429782032966614, z
    =0.014744692512349077, roll=7.884650441866775e-28, pitch=-7.554679105908491
    e-28, yaw=2.245637386214381)
table: Pose=RavenPose(x=0.0, y=-0.5, z=0.0, roll=0.0, pitch=-0.0, yaw=0.0)
Task goal: serve the banana inside the blue thing after drying it by placing on the
plate

```

```

#define assistant
'''python
def gen_plan(initial:RavenState):
    plan = []
    place_pose = PlaceSampler(initial, env, "plate").sample(rng)
    banana_grasp = GraspSampler(initial, env, "banana").sample(rng)
    plan.append(Action("pick", [[banana], grasp]))
    plan.append(Action("place_ontop", [[banana, plate], place_pose]))
    bowl_bounds = get_aabb_bounds(initial, env, "bowl")
    place_pose = PlaceSampler(initial, env, "bowl").sample(rng)
    bowl_center_pose = get_obj_center(initial, env)
    # Drop the object in the exact center of the bowl
    place_pose[0] = bowl_center_pose[0]
    place_pose[1] = bowl_center_pose[1]
    place_pose[2] = bowl_center_pose + 0.05
    # Make sure to pick an orientation such that the banana will fit!
    place_pose.roll = 1.5
    banana_grasp = GraspSampler(initial, env, "banana").sample(rng)
    plan.append(Action("pick", [[banana], grasp]))
    plan.append(Action("place_inside", [[banana, bowl], place_pose]))
    return plan
'''

```

Here is yet another example for a different task:

```

mug: Pose=RavenPose(x=-0.09269248694181442, y=-0.7042829990386963, z
    =0.026169249787926674, roll=0.0, pitch=-0.0, yaw=0.8605557025412023)
potted_meat_can: Pose=RavenPose(x=0.17416073374449514, y=-0.33348321026557554, z
    =0.02017684663429707, roll=5.081222700168695e-05, pitch=0.00013538346655467005,
    yaw=-3.0371082921616765)
plate: Pose=RavenPose(x=-0.11636300384998322, y=-0.4429782032966614, z
    =0.014744692512349077, roll=7.884650441866775e-28, pitch=-7.554679105908491
    e-28, yaw=2.245637386214381)
table: Pose=RavenPose(x=0.0, y=-0.5, z=0.0, roll=0.0, pitch=-0.0, yaw=0.0)
Task goal: serve spam from its can into the cup

```

```

#define assistant
'''python
def gen_plan(initial:RavenState):
    plan = []

```

```

spam_grasp = GraspSampler(initial, env, "potted_meat_can").sample(rng)
pour_params = PourSampler(initial, env, "potted_meat_can", "cup").sample(rng)
pour_params[3] = 2.0 # necessary for the contents to really fall into the cup.
plan.append(Action("pick", [[potted_meat_can], spam_grasp]))
plan.append(Action("pour", [[potted_meat_can, mug], pour_params]))
return plan
'''

```

Make sure to enclose your output with `python <output gen_plan(initial:Ravenstate)`
`: function> '''` (ignore the angle brackets - those are just for illustrative
purposes).